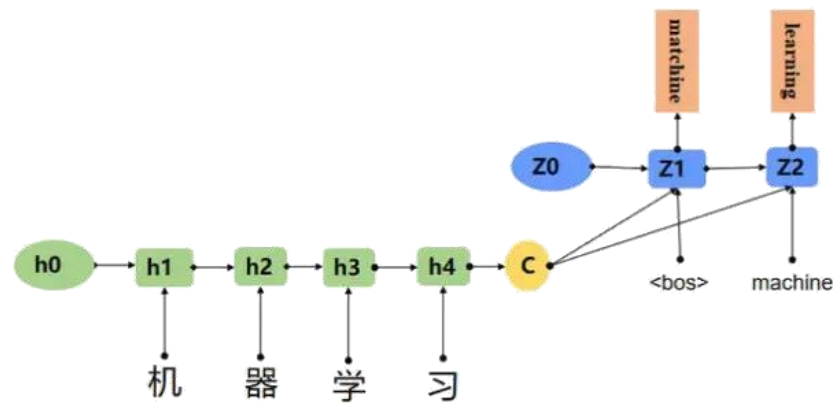


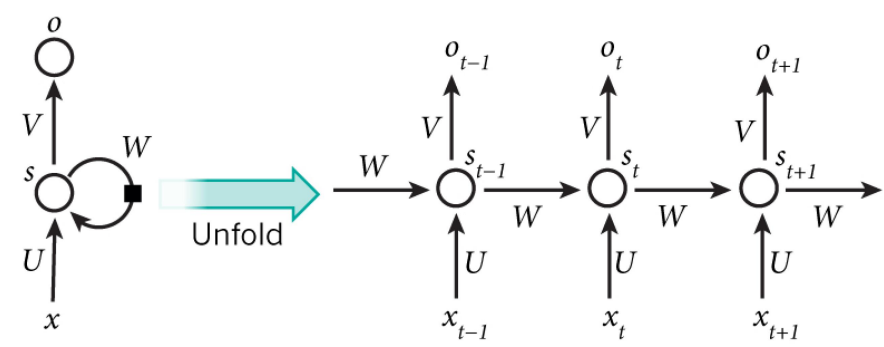
Transformer、 Bert

Why Transformer ?

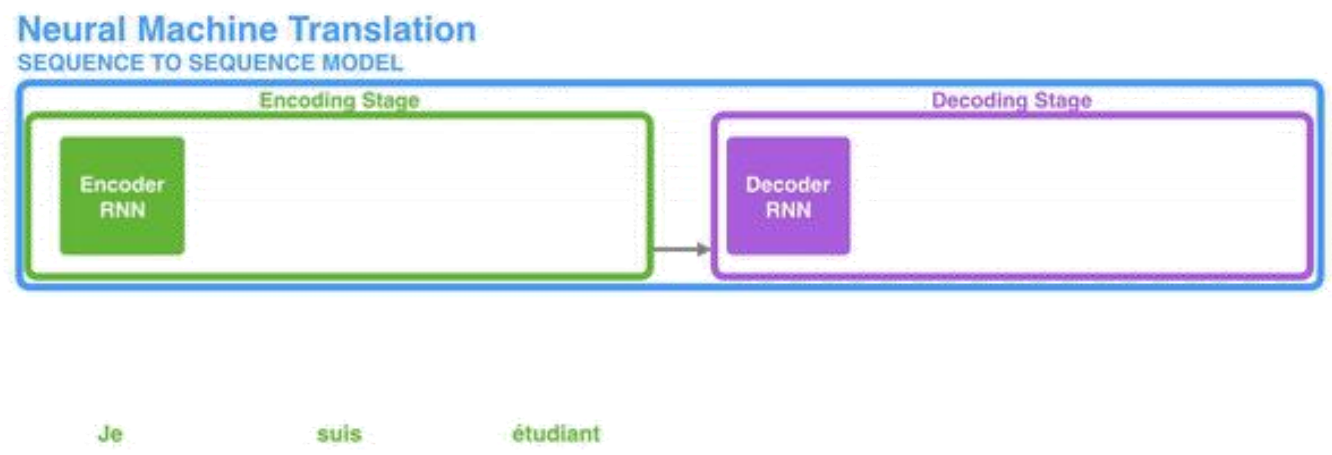
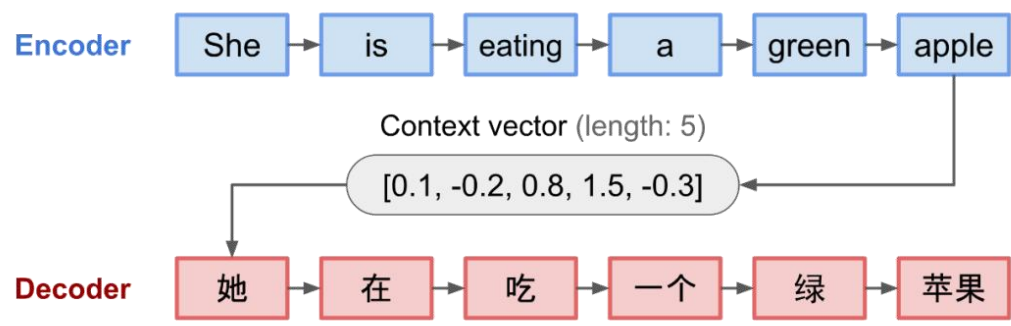
Seq2Seq 任务示例 (NMT, 神经机器翻译)



Seq2Seq模型示例——RNN

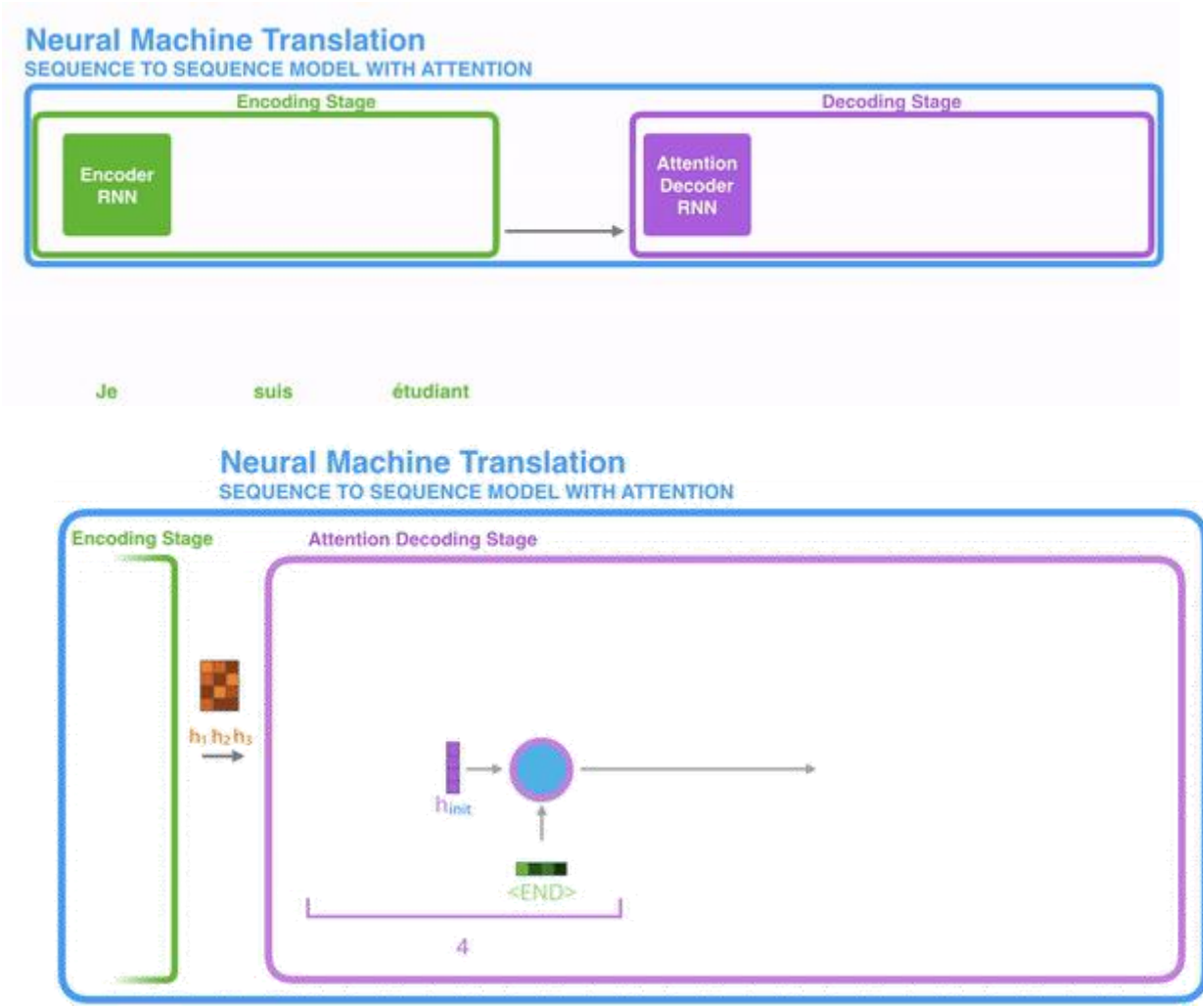


Encoder-Decoder 框架 (编码器+解码器)



Why Transformer ?

Seq2Seq model with Attention



RNN及其变体模型无法并行计算，模型效率低下！

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

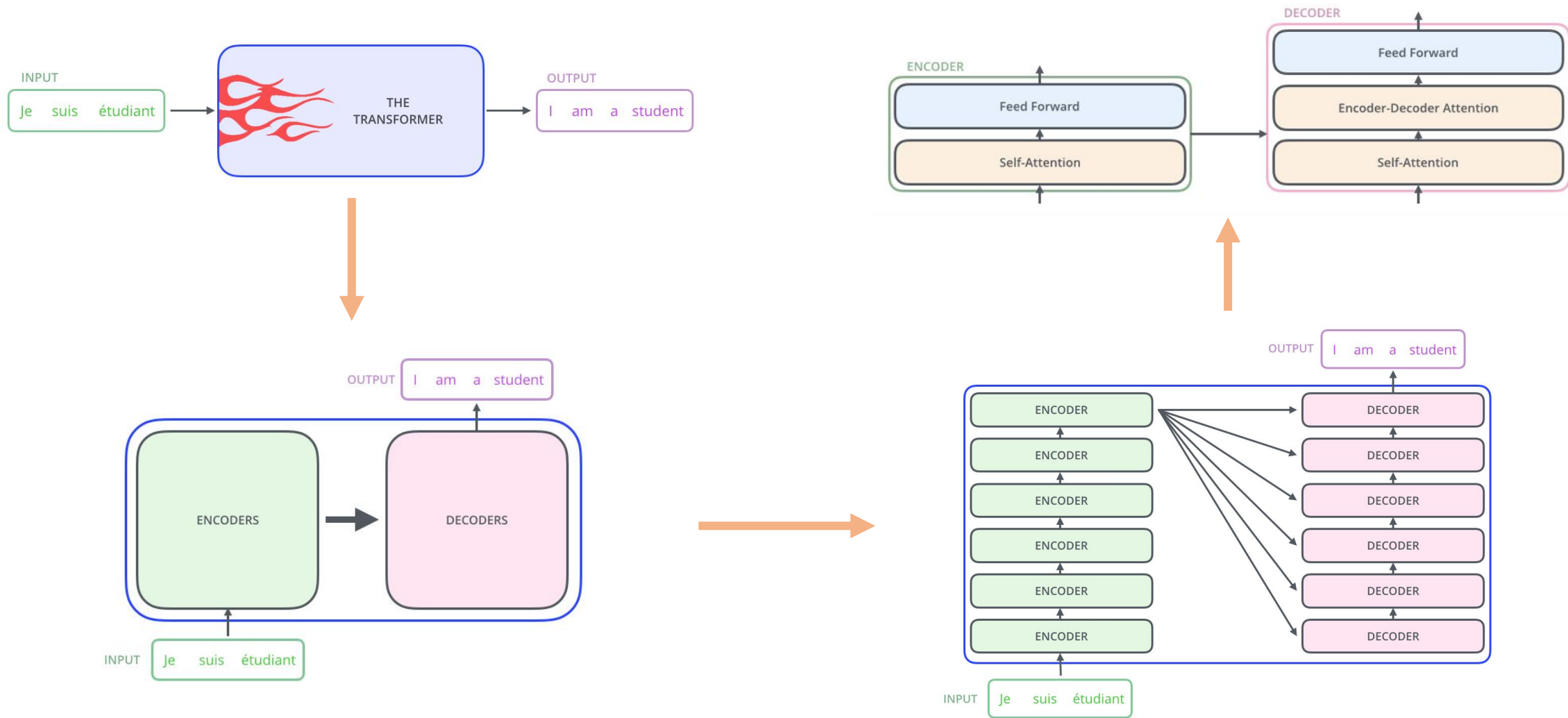
Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

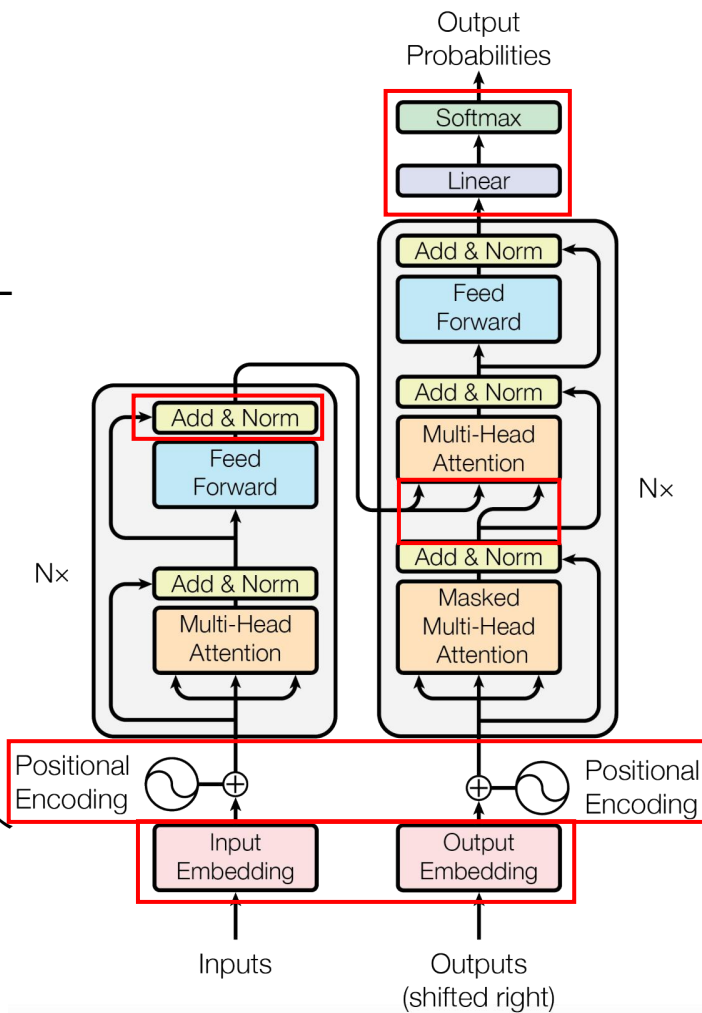
Transformer 架构



Transformer 架构

编码器

- 由N个block堆叠而成;
- 每个block有两层:
 - Multi-Head Attention (Self-Attention)
 - + Add (Residual Connection)
 - + Norm (LayerNorm);
 - Feed Forward
 - + Add (Residual Connection)
 - + Norm (LayerNorm);
- Block₁ ~ Block_{N-1}的输出: 输入到下个Block;
- Block_N的输出: 输入到解码器的各层中。



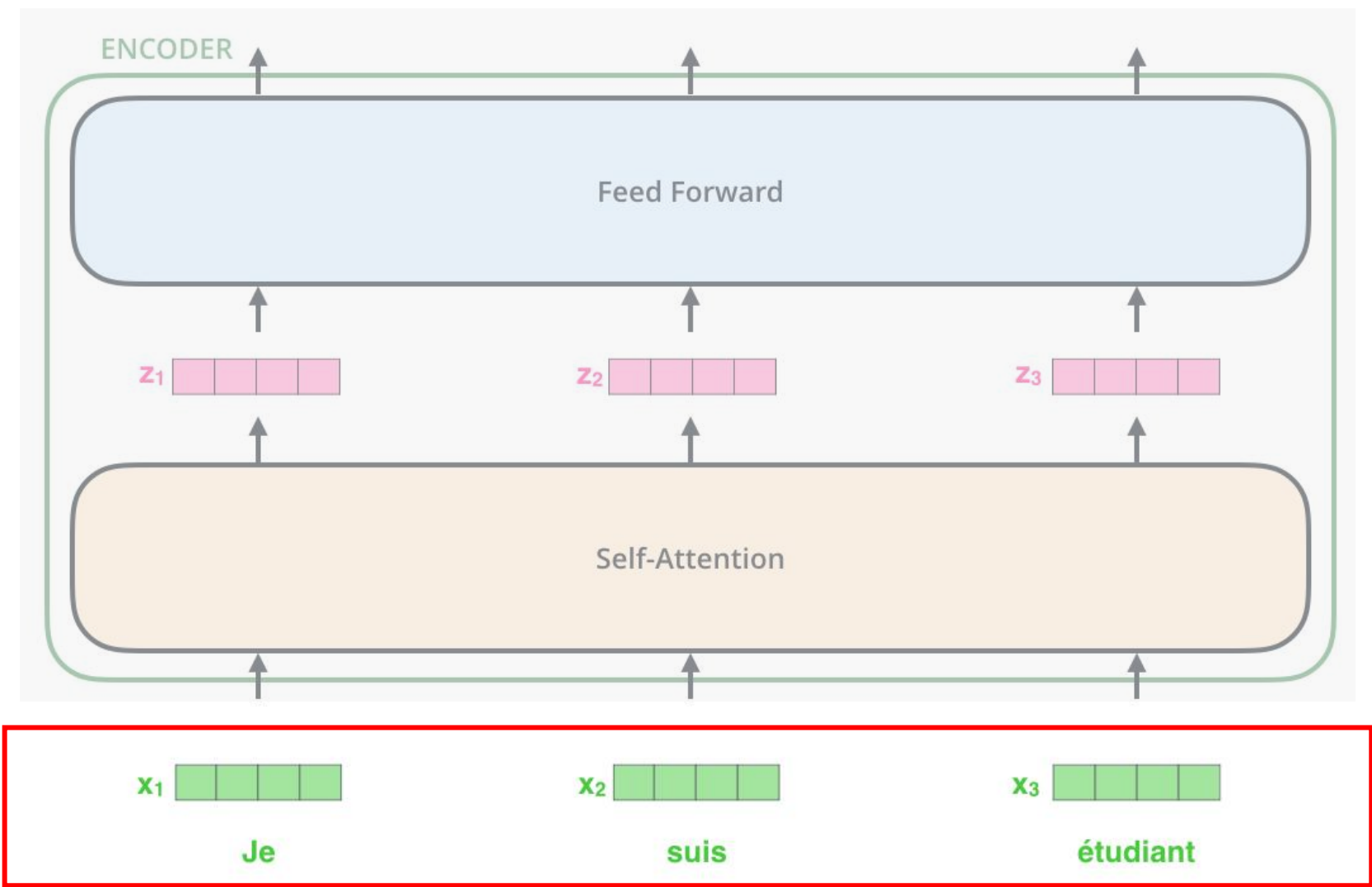
解码器

- 由N个block堆叠而成;
- 每个block有三层:
 - Masked Multi-Head Attention (Self-Attention)
 - + Add (Residual Connection)
 - + Norm (LayerNorm);
 - Multi-Head Attention (Co-Attention)
 - + Add (Residual Connection)
 - + Norm (LayerNorm);
 - Feed Forward
 - + Add (Residual Connection)
 - + Norm (LayerNorm);
- Block₁ ~ Block_{N-1}的输出: 输入到下个Block;
- Block_N的输出: 输入到后续的Linear层中。

Transformer 工作流程

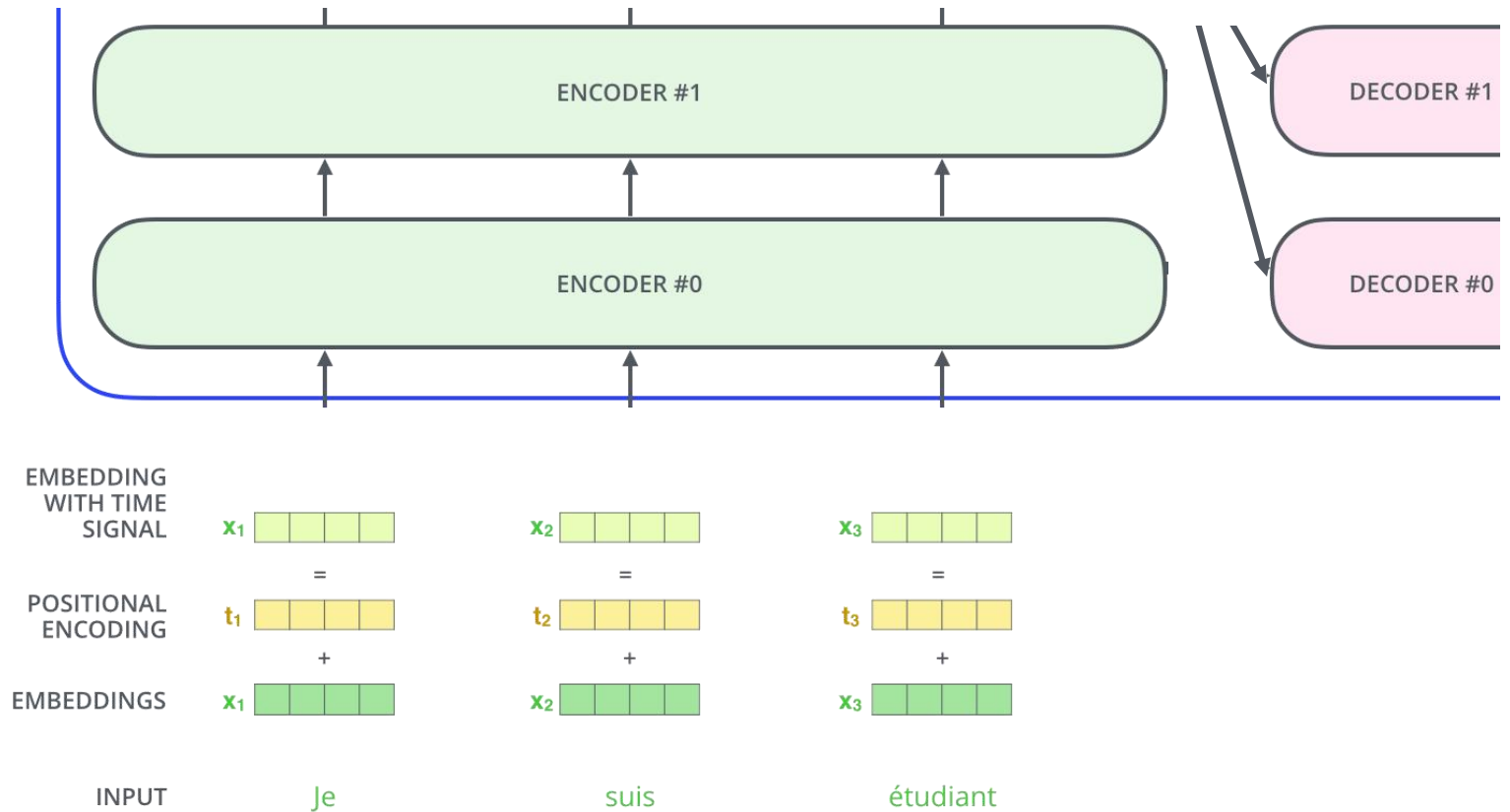
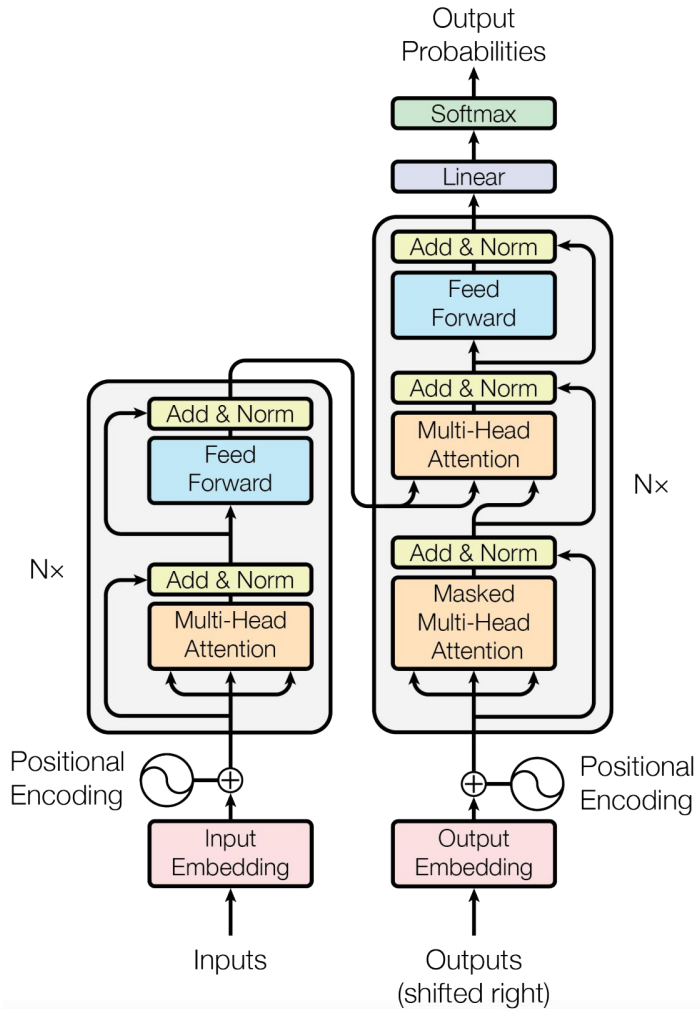
Word

Embedding



Transformer 工作流程

位置编码 (Positional Encoding)



Transformer 工作流程

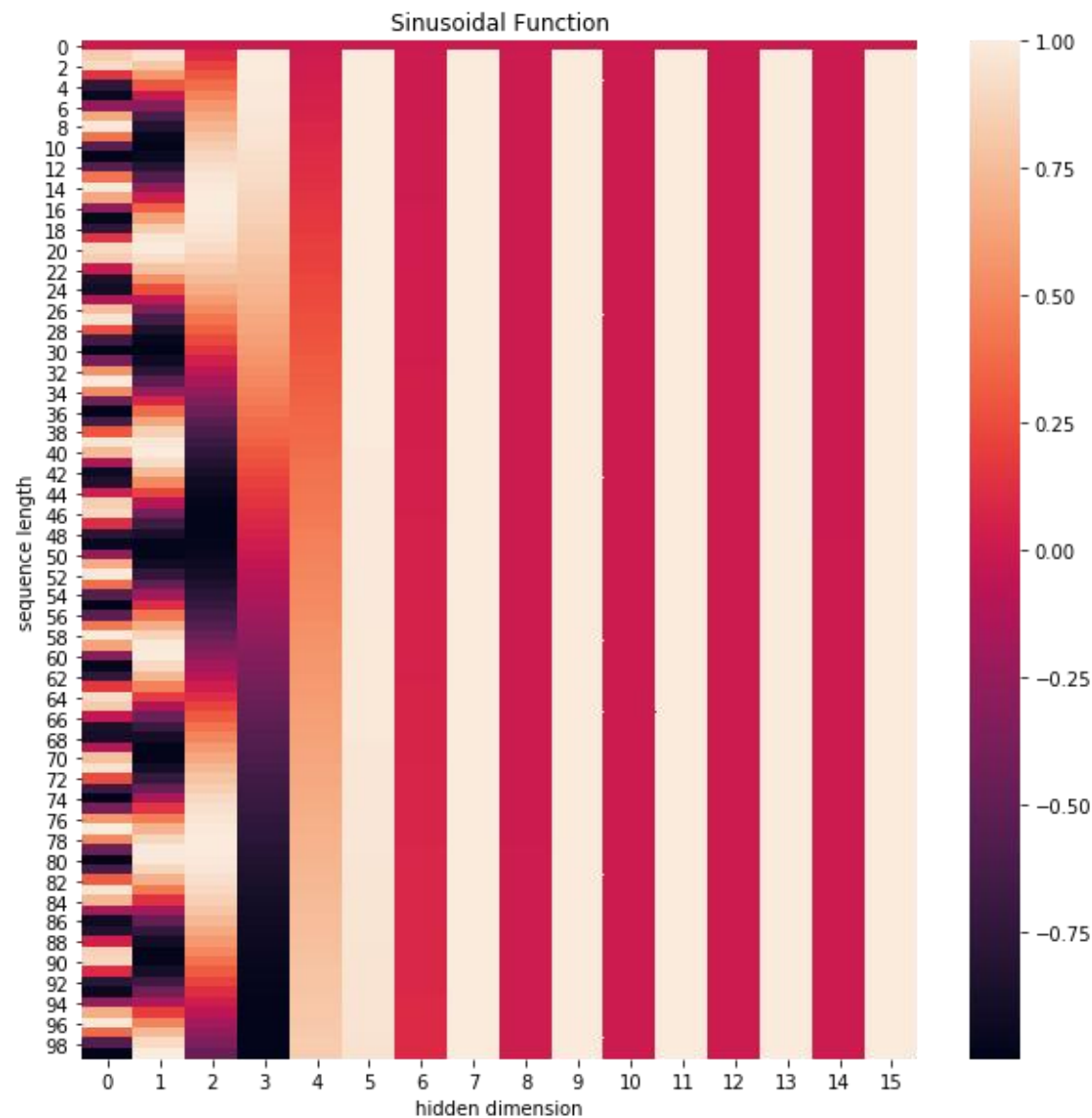
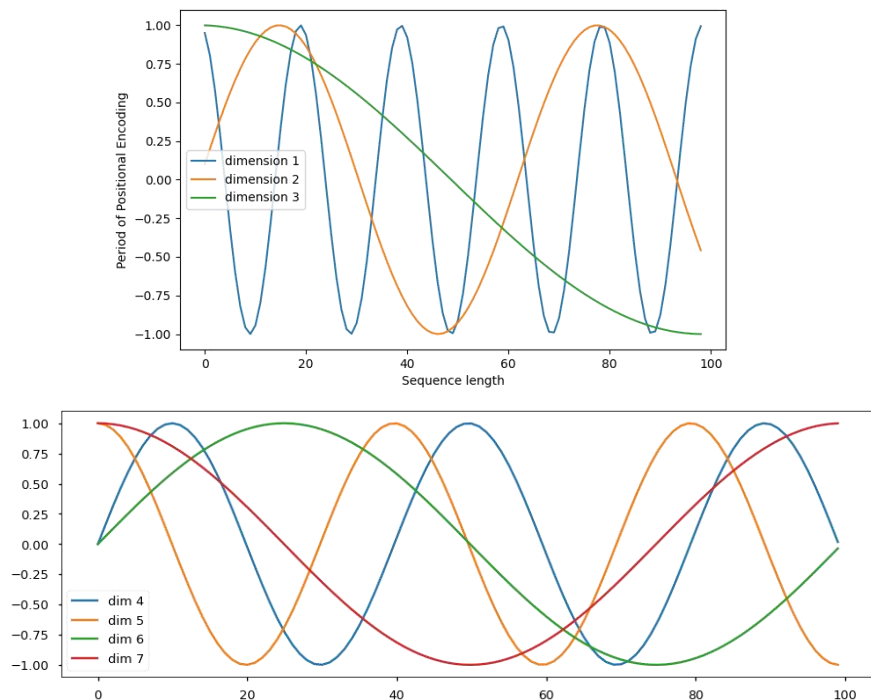
位置编码 (Positional Encoding)

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}})$$
$$\text{PE}(\text{pos}, 2i + 1) = \cos(\text{pos}/10000^{2i/d_{\text{model}}})$$

$\text{pos} \in [0, \text{max_sequence_length})$

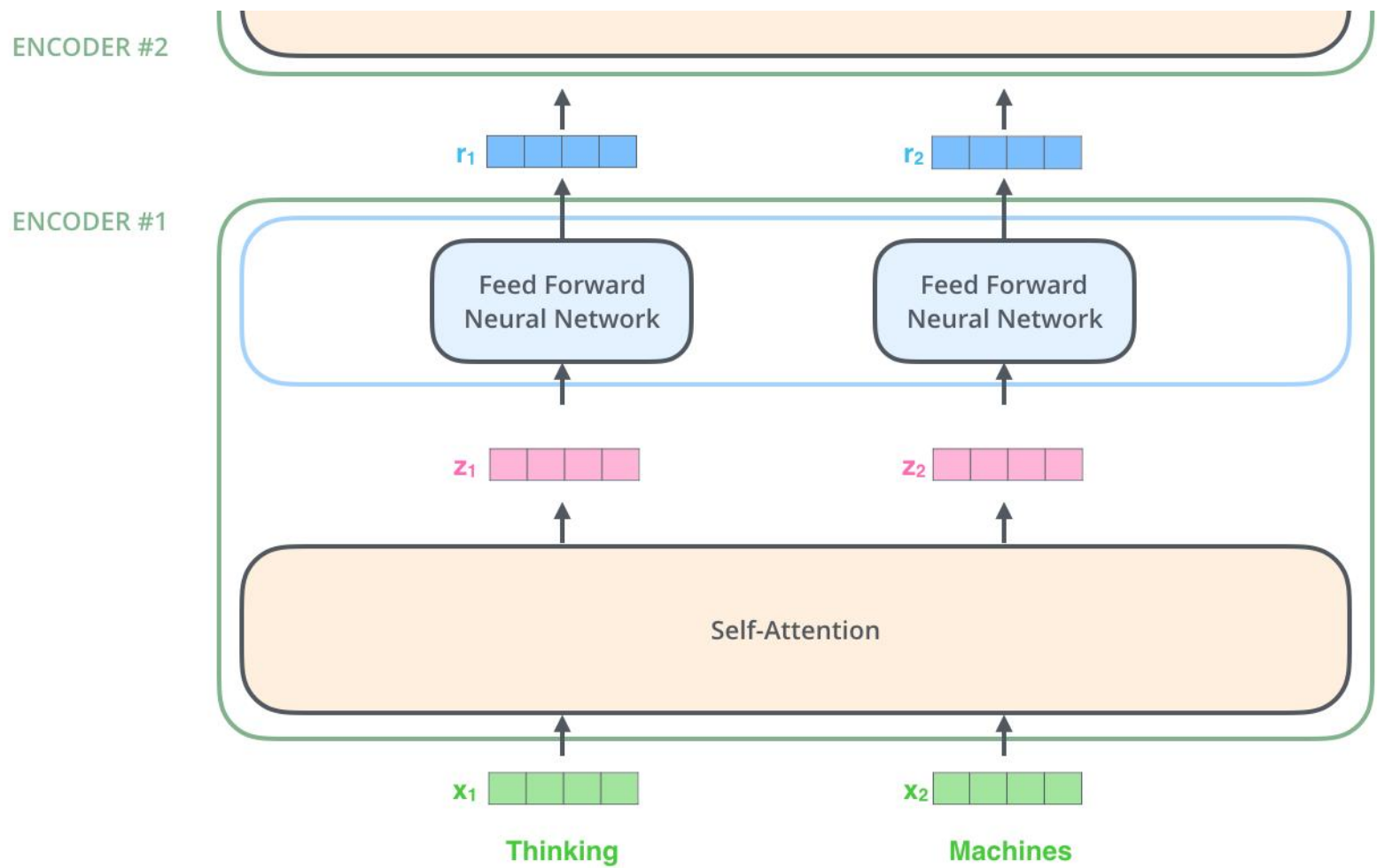
$i \in [0, \frac{d_{\text{model}}}{2})$

$$\tau = \frac{2\pi}{\omega} = 2\pi * 10000^{\frac{2i}{d_{\text{model}}}}$$



Transformer 工作流程

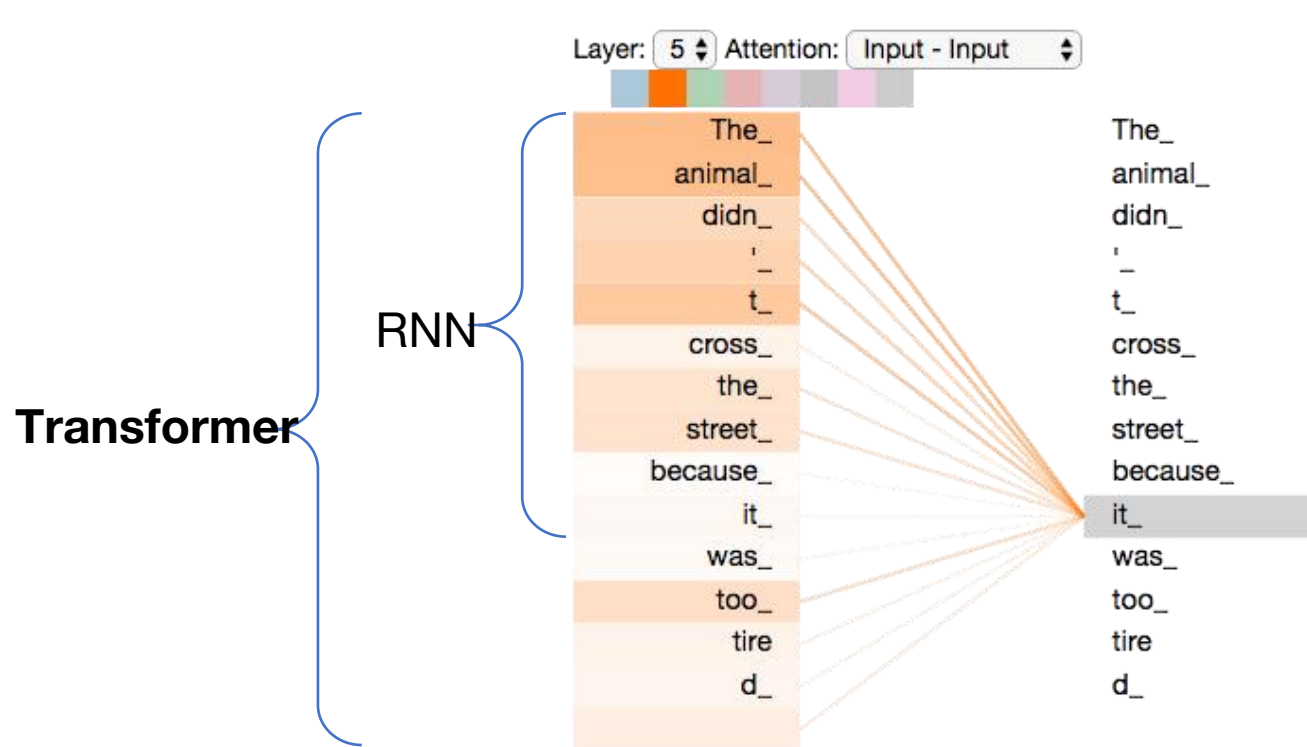
编码过程



Transformer 工作流程

编码过程 —— Self-Attention (宏观)

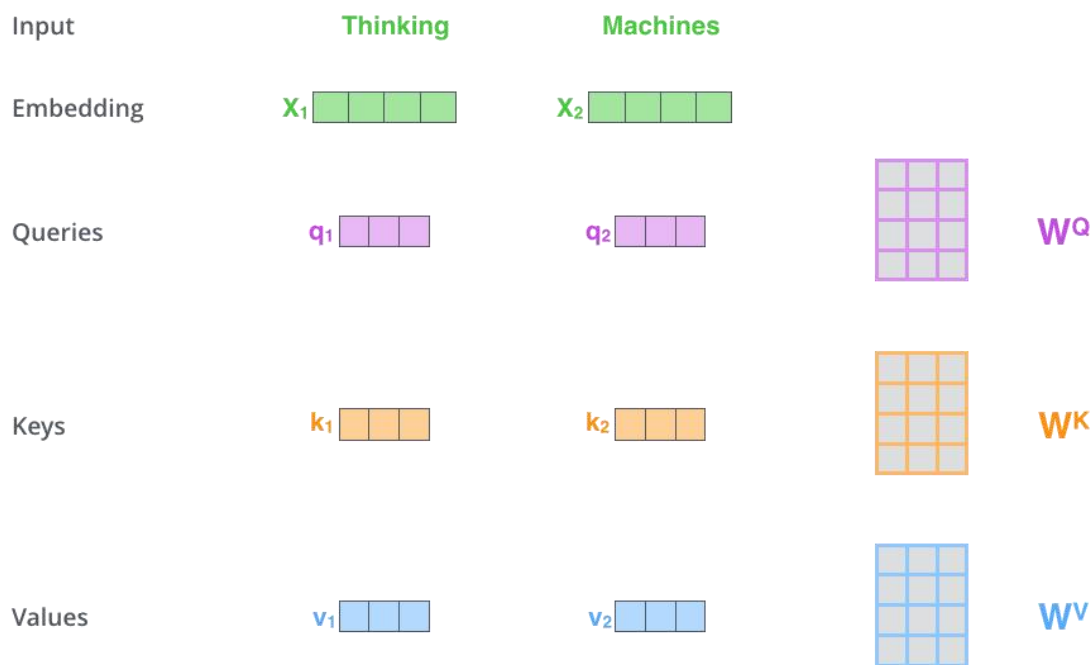
The animal didn't cross the street because it was too tired.



Transformer 工作流程

编码过程 —— Self-Attention (微观)

第一步：生成 Q、K、V，辅助计算注意力机制



X_1 与 W^Q 权重矩阵相乘得到 q_1 ，就是与这个单词相关的查询向量。通过这种方式，为输入序列的每个单词都创建一个查询向量Q、一个键向量K和一个值向量V。

Transformer 工作流程

编码过程 —— Self-Attention (微观)

核心公式: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$

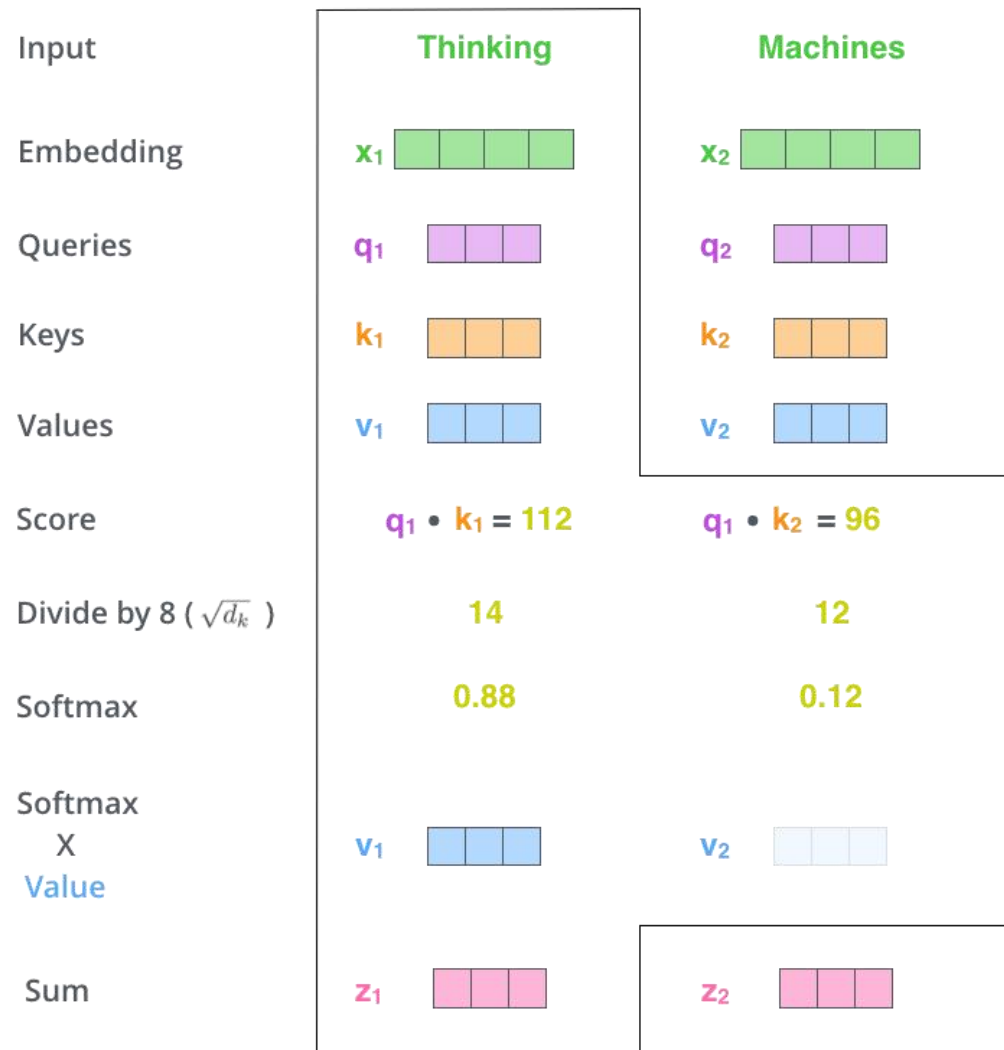
第二步: 计算当前单词的Q与候选单词的K的点积:

第三步: 将上一步结果除以维度的平方根:

第四步: 将上一步结果通过softmax函数转换:

第五步: 将候选单词的每个值向量乘以softmax分数:

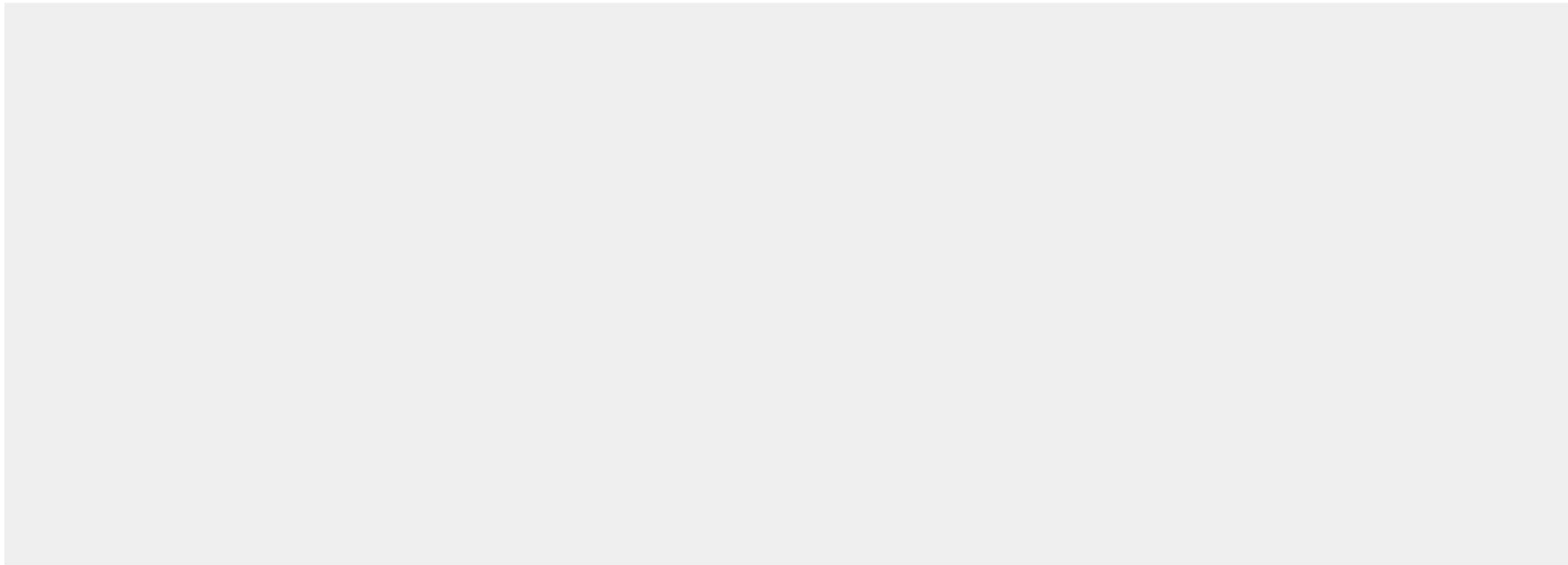
第六步: 对加权后的值向量求和,
即得到自注意力层在该位置的输出:



Transformer 工作流程

编码过程 — Self-Attention (微观)

Self-attention



input #1

1	0	1	0
---	---	---	---

input #2

0	2	0	2
---	---	---	---

input #3

1	1	1	1
---	---	---	---

Transformer 工作流程

编码过程 —— Self-Attention (微观)

通过矩阵运算实现自注意力机制

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{Q}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{K}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{K} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$

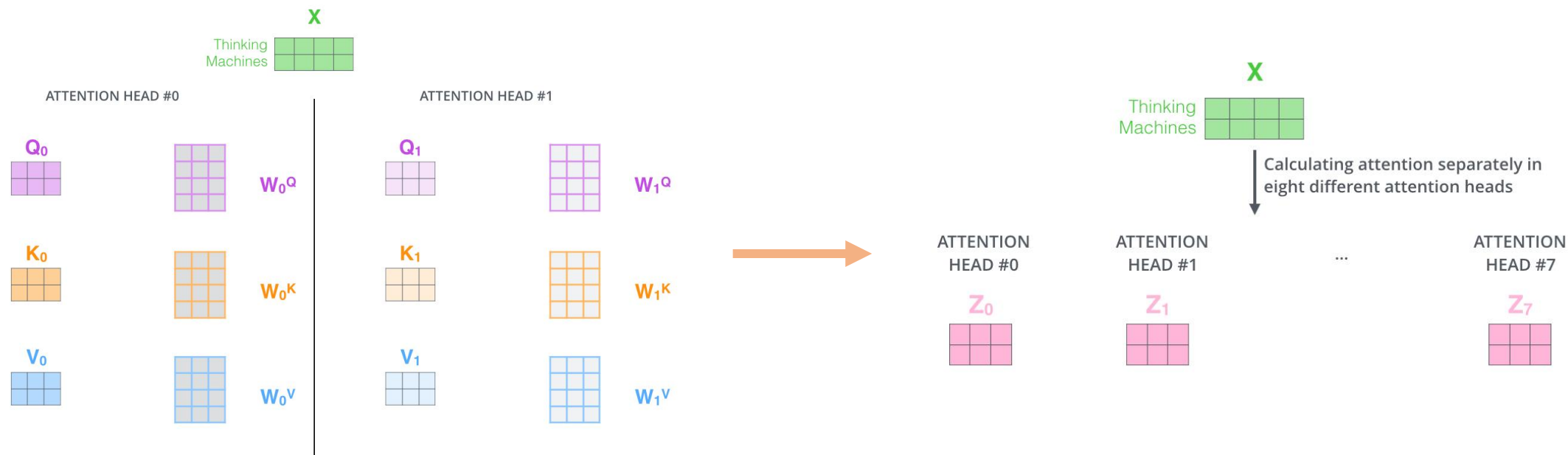
$$\begin{matrix} \text{X} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{W}^{\text{V}} \\ \begin{array}{|c|c|c|c|} \hline \square & \square & \square & \square \\ \hline \square & \square & \square & \square \\ \hline \end{array} \end{matrix} = \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}$$



$$\begin{aligned} & \text{softmax} \left(\frac{\begin{matrix} \text{Q} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \times \begin{matrix} \text{K}^{\text{T}} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix}}{\sqrt{d_k}} \right) \begin{matrix} \text{V} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \\ & = \begin{matrix} \text{Z} \\ \begin{array}{|c|c|c|} \hline \square & \square & \square \\ \hline \square & \square & \square \\ \hline \end{array} \end{matrix} \end{aligned}$$

Transformer 工作流程

编码过程 — Multi-Head Attention



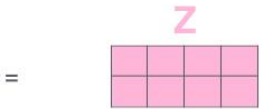
Transformer 工作流程

编码过程 — Multi-Head Attention

1) Concatenate all the attention heads

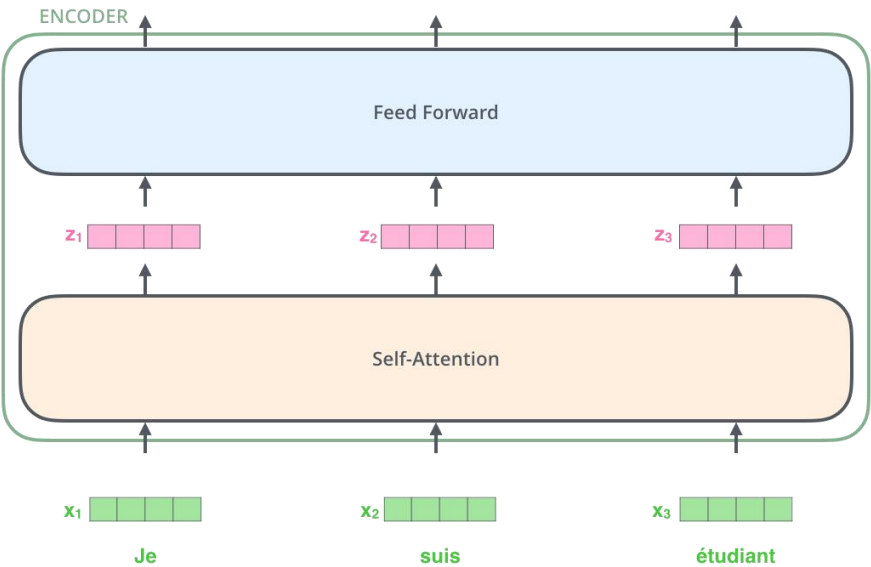
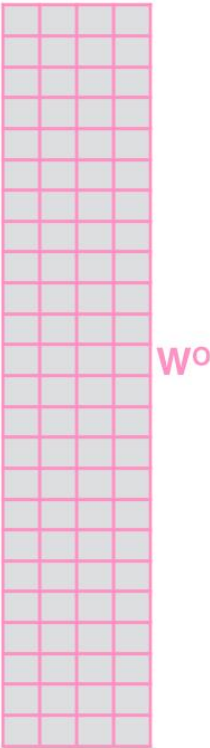


3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN



2) Multiply with a weight matrix W^O that was trained jointly with the model

X



Transformer 工作流程

编码过程 — — Multi-Head Attention

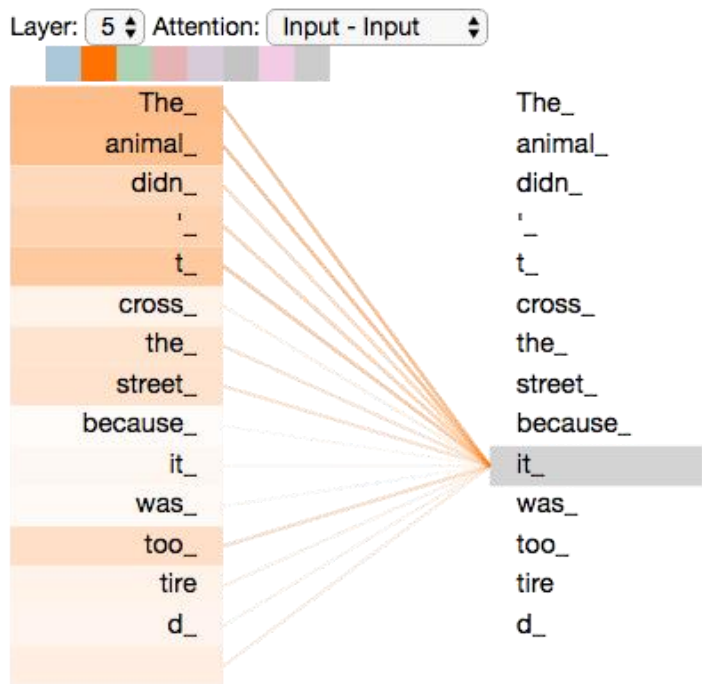
1) This is our
input sentence*

Thinking
Machines

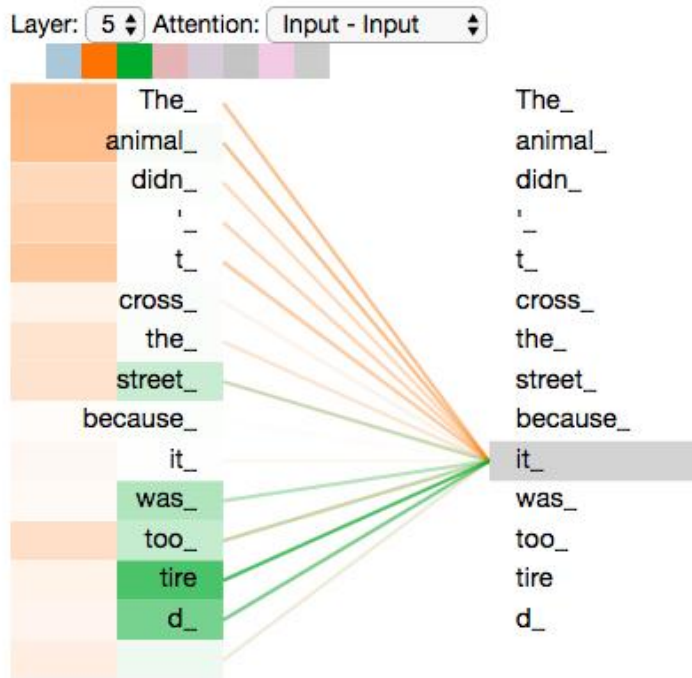
Transformer 工作流程

编码过程 — Multi-Head Attention

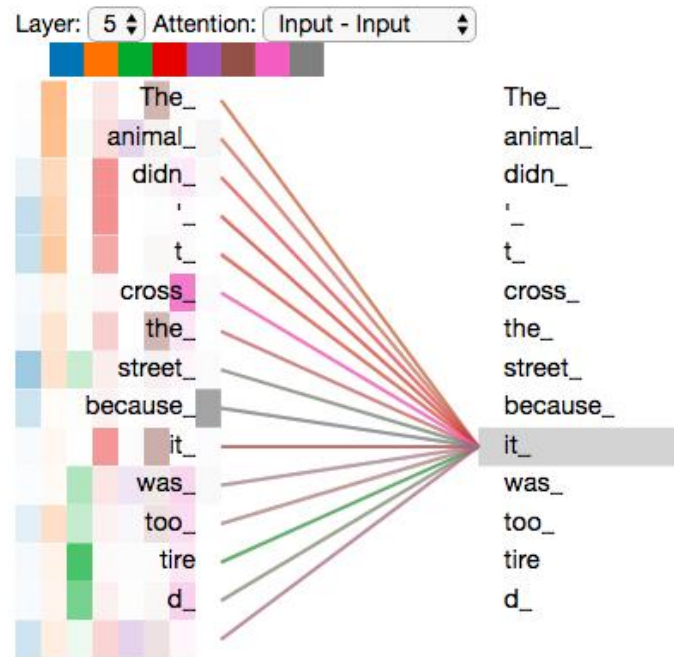
One-Head



Two-Heads



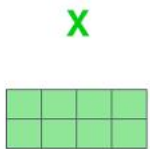
All-Heads



Transformer 工作流程

Padding 操作

X: Thinking Machines

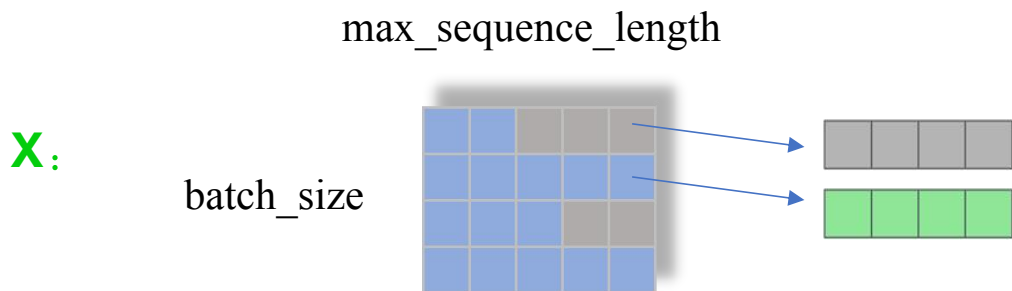


X 的维度: [sequence_length, embedding_dimension]

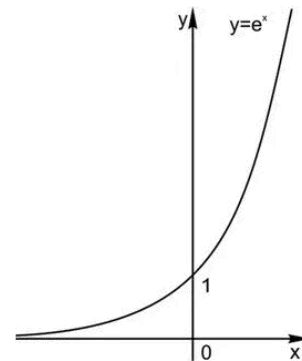
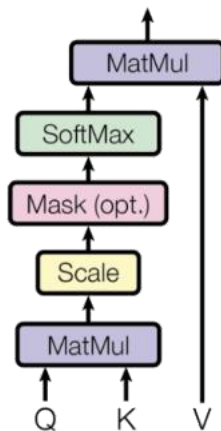


X: Thinking Machines (seq_len: 2)
A Tale of Two Cities (seq_len: 5)
Science and Art (seq_len: 3)
the Art of Motorcycle Maintenance (seq_len: 5)

X 的维度: [batch_size, max_sequence_length, embedding_dimension]



Scaled dot-product attention



Softmax函数: $\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$

$e^0 = 1$
 $e^{-\infty} \rightarrow 0$

Padding Mask

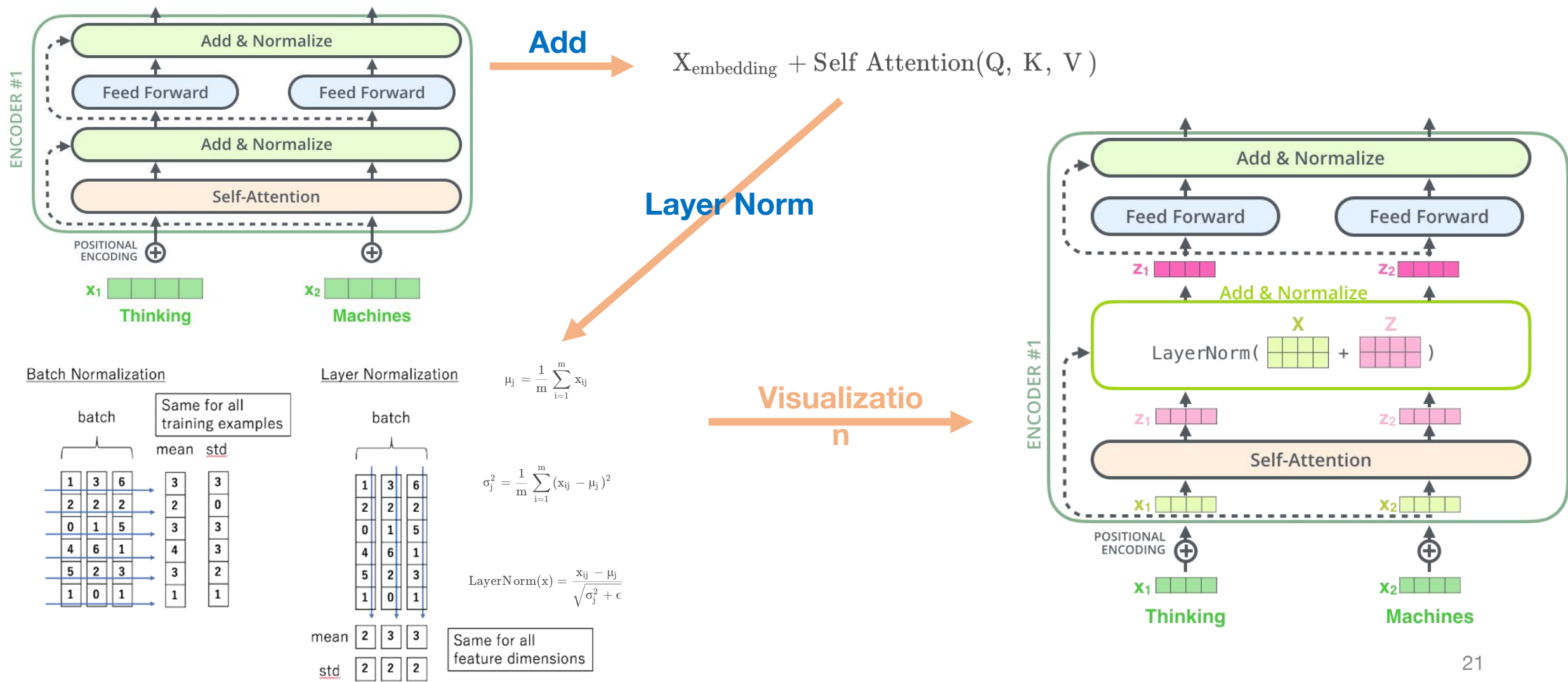
0	0	1	1	1
0	0	0	0	0
0	0	0	1	1
0	0	0	0	0



0	0	-inf	-inf	-inf
0	0	0	0	0
0	0	0	-inf	-inf
0	0	0	0	0

Transformer 工作流程

Add & Norm



Transformer 工作流程

Add & Norm

$$X_{\text{hidden}} = X_{\text{attention}} + X_{\text{hidden}}$$
$$X_{\text{hidden}} = \text{LayerNorm}(X_{\text{hidden}})$$

$$X_{\text{hidden}} = \text{Linear}(\text{ReLU}(\text{Linear}(X_{\text{attention}})))$$

$$X_{\text{attention}} = X + X_{\text{attention}}$$
$$X_{\text{attention}} = \text{LayerNorm}(X_{\text{attention}})$$

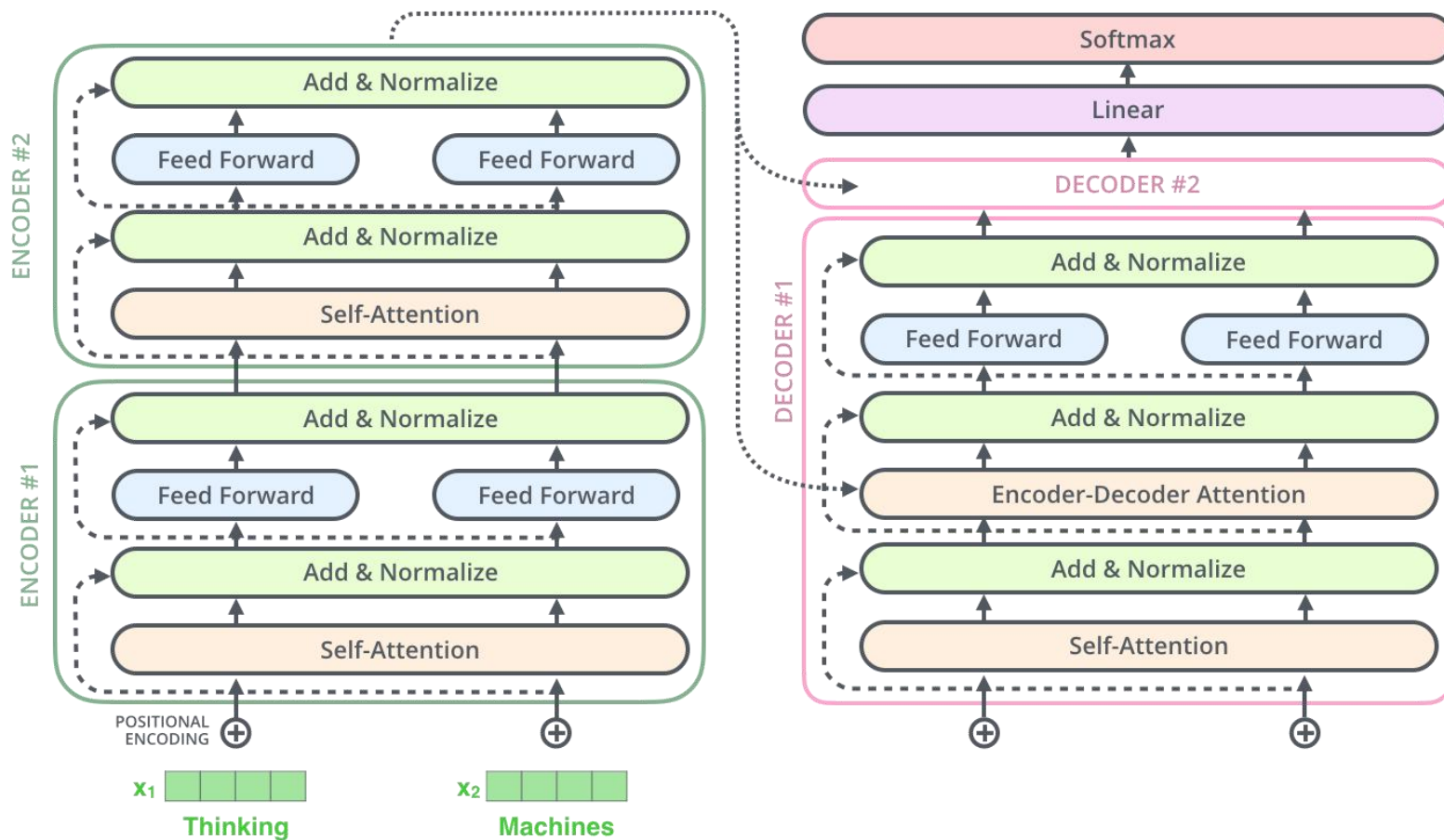
$$Q = \text{Linear}(X) = XW_Q$$

$$K = \text{Linear}(X) = XW_K$$

$$V = \text{Linear}(X) = XW_V$$

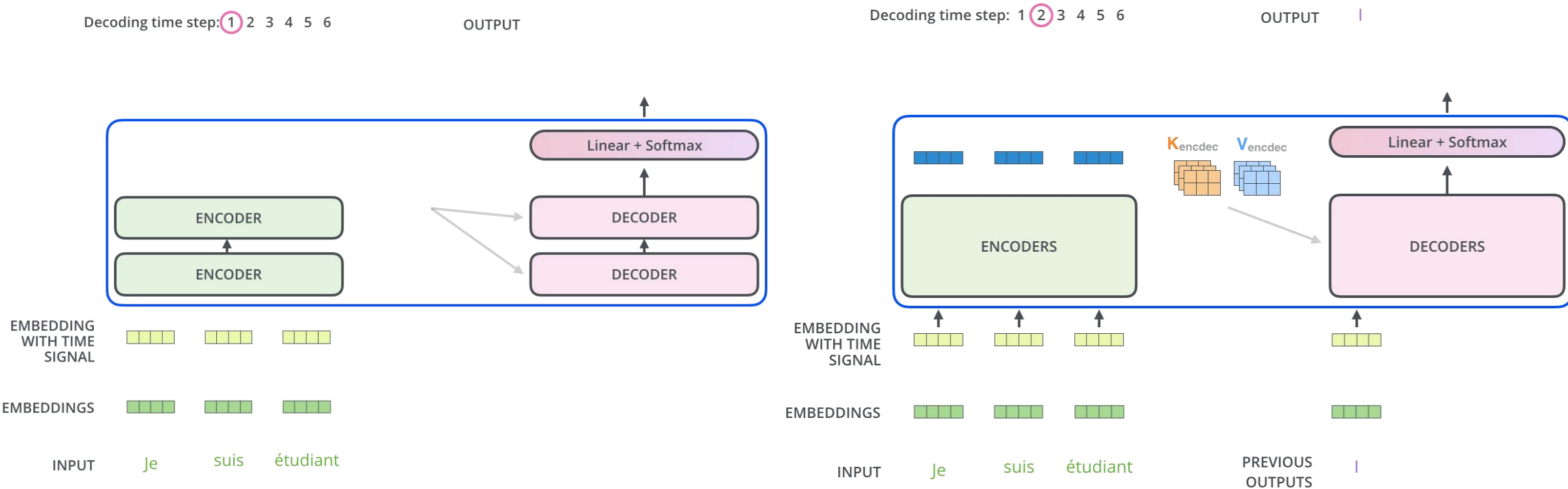
$$X_{\text{attention}} = \text{SelfAttention}(Q, K, V)$$

$$X = \text{Embedding Lookup}(X) + \text{Positional Encoding}$$



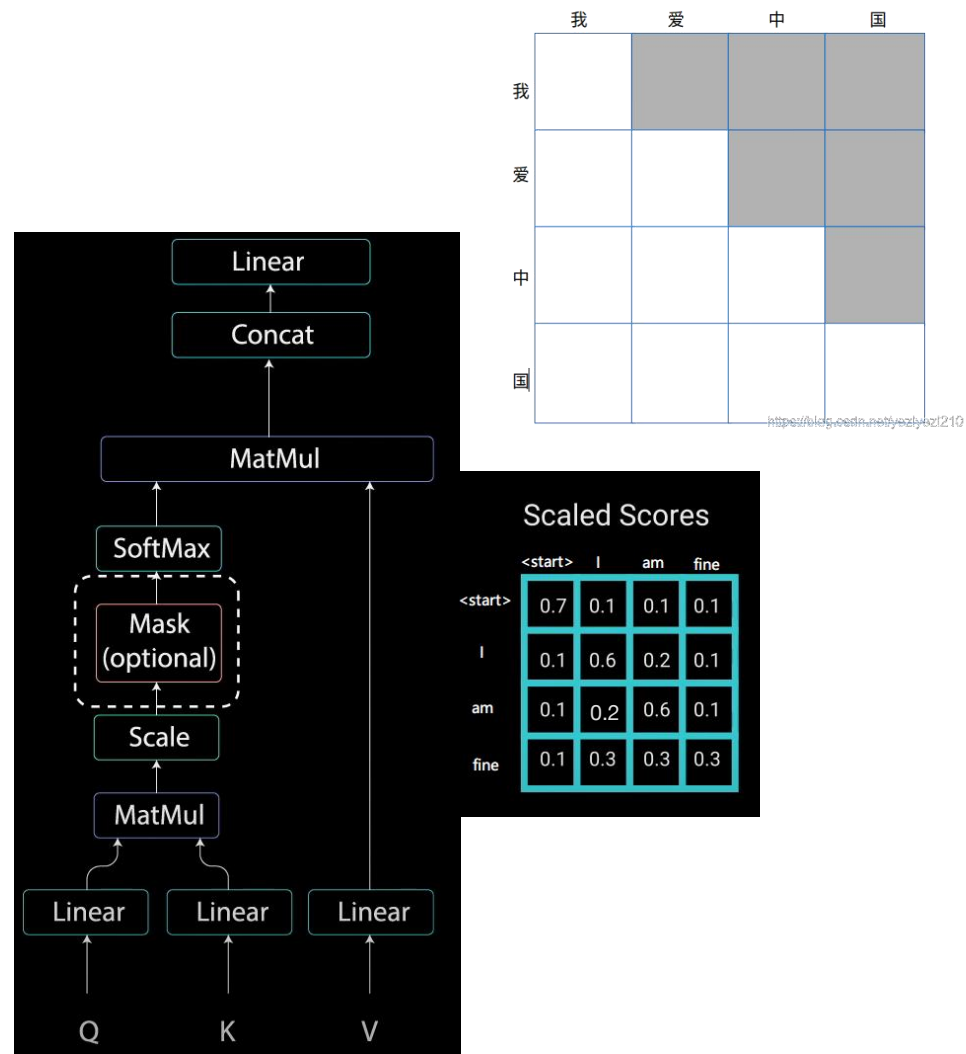
Transformer 工作流程

解码过程 — Multi-Head Attention

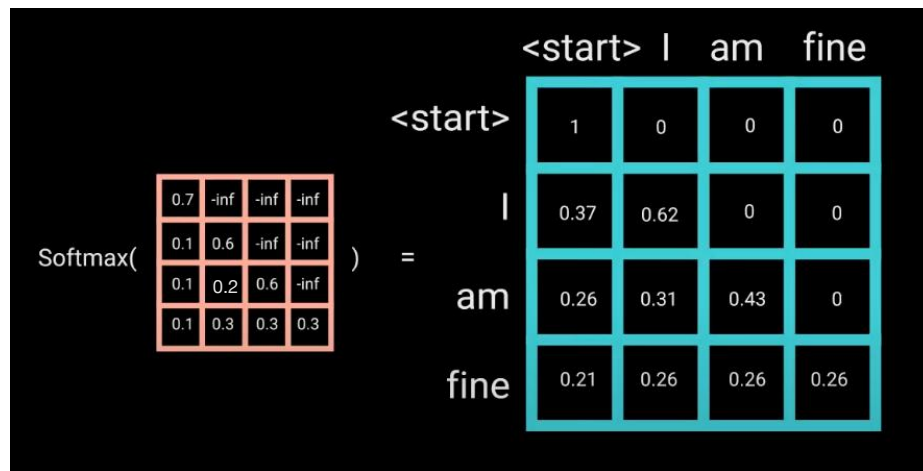
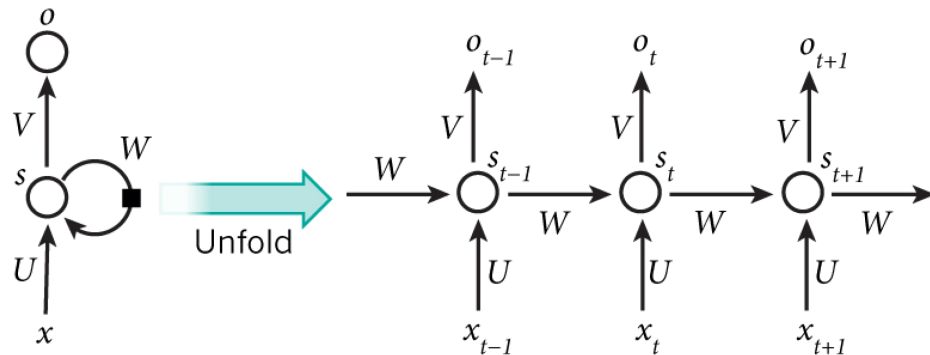


Transformer 工作流程

解码过程 — Masked Self-Attention



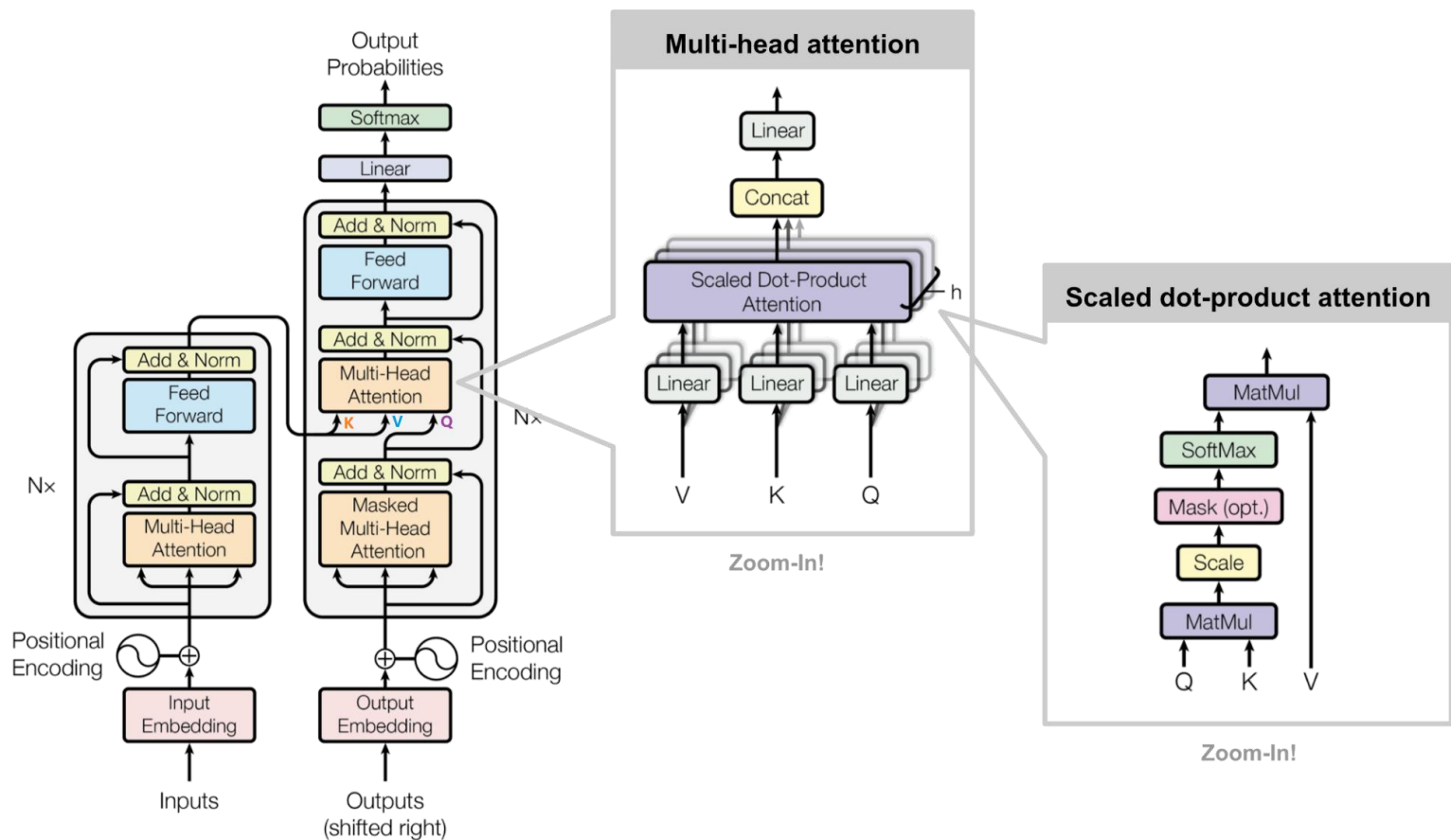
RNN模型:



Transformer 工作流程

解码过程 — Masked Encoder-Decoder

Attention



- **Encoder Multi-Head Attention:**

- Q, K, V all from Encoders

- **Decoder Masked Multi-Head Attention:**

- Q, K, V all from Decoders

- **Decoder Multi-Head Attention:**

- Q from Decoders
- K, V from Encoders

Transformer 工作流程

解码过程 — Linear & Softmax

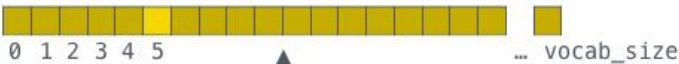
Which word in our vocabulary
is associated with this index?

am

Get the index of the cell
with the highest value
(argmax)

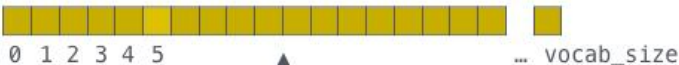
5

log_probs



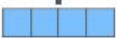
Softmax

logits



Linear

Decoder stack output



Transformer 训练

Output Vocabulary

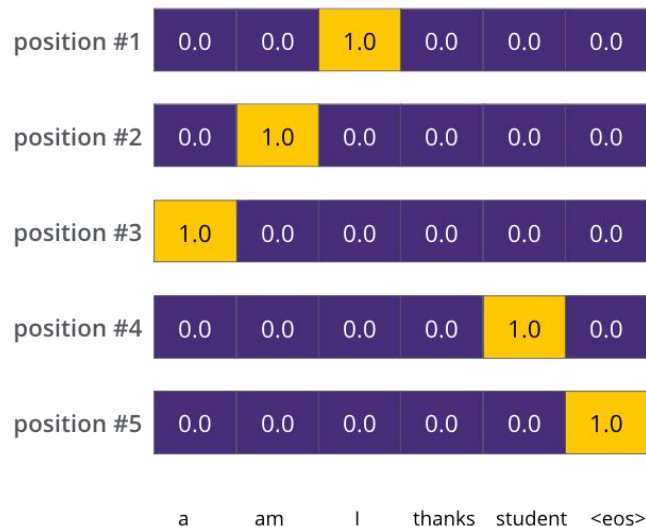
WORD	a	am	i	thanks	student	<eos>
INDEX	0	1	2	3	4	5

→ eos: end of sentence的缩写形式

输入“je suis étudiant”，期望输出是“i am a student”

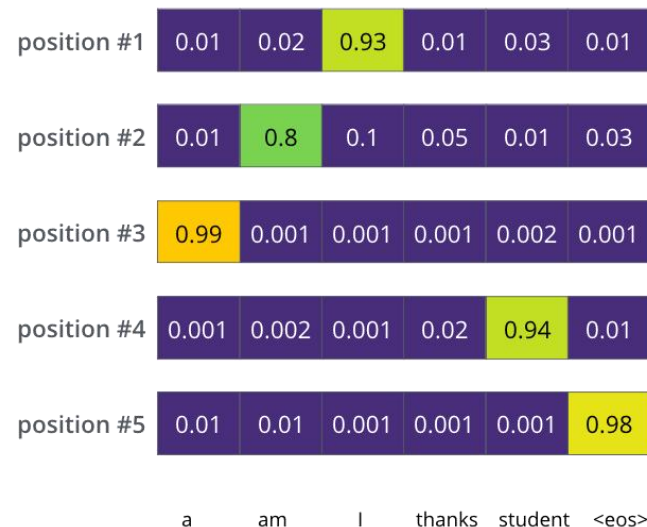
Target Model Outputs

Output Vocabulary: a am i thanks student <eos>



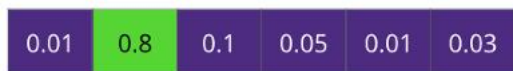
Trained Model Outputs

Output Vocabulary: a am i thanks student <eos>



Transformer 预测

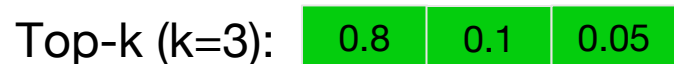
解码/采样策略



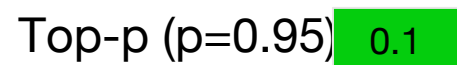
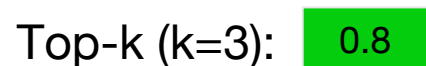
概率降序排列



缩小采样范围



随机采样



Timestep	1	2	3	4
A	0.5	0.1	0.2	0.0
B	0.2	0.4	0.2	0.2
C	0.2	0.3	0.4	0.2
<eos>	0.1	0.2	0.2	0.6

Argmax Decoding (贪婪采样)

Greedy Search (贪心搜索)

Beam Search (束束搜索)

Stochastic Decoding (随机采样)

Temperature-controlled Stochastic Sampling

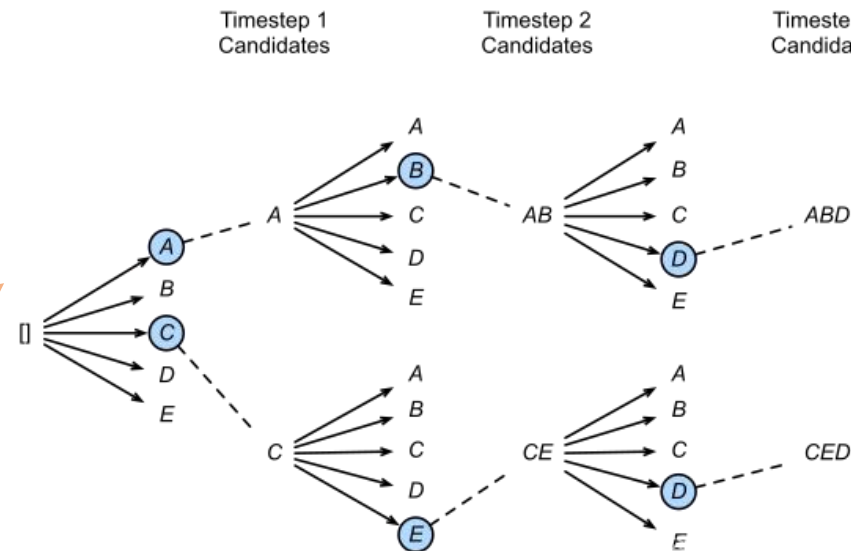
Top-k Sampling

Top-p Sampling (Nucleus Sampling)

混合采样

CTRL:
$$p_i = \frac{\exp(x_i / (T \cdot I(i \in g)))}{\sum_j \exp(x_j / (T \cdot I(j \in g)))}$$

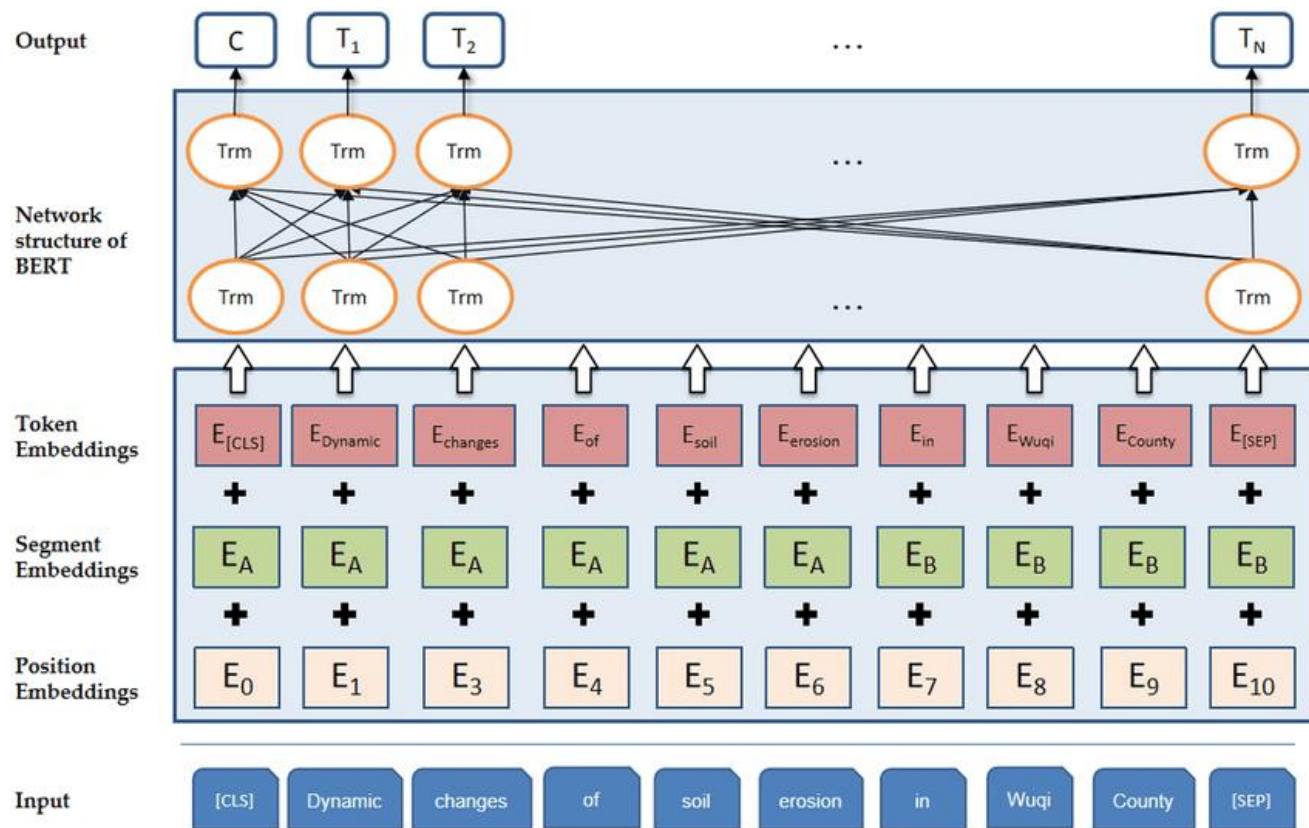
$$I(c) = \theta \text{ if } c \text{ is True else } 1$$



$$P(x|x_{1:t-1}) = \frac{\exp(u_t/t)}{\sum_{t'} \exp(u_{t'}/t)}, \text{ 其中 } t \in [0, 1)$$

BERT

- BERT架构：一种基于多层Transformer编码器的预训练语言模型，通过结合Tokenization、多种Embeddings和特定任务的输出层，能够捕捉文本的双向上下文信息，并在各种自然语言处理任务中表现出色。



输入层 (Input)

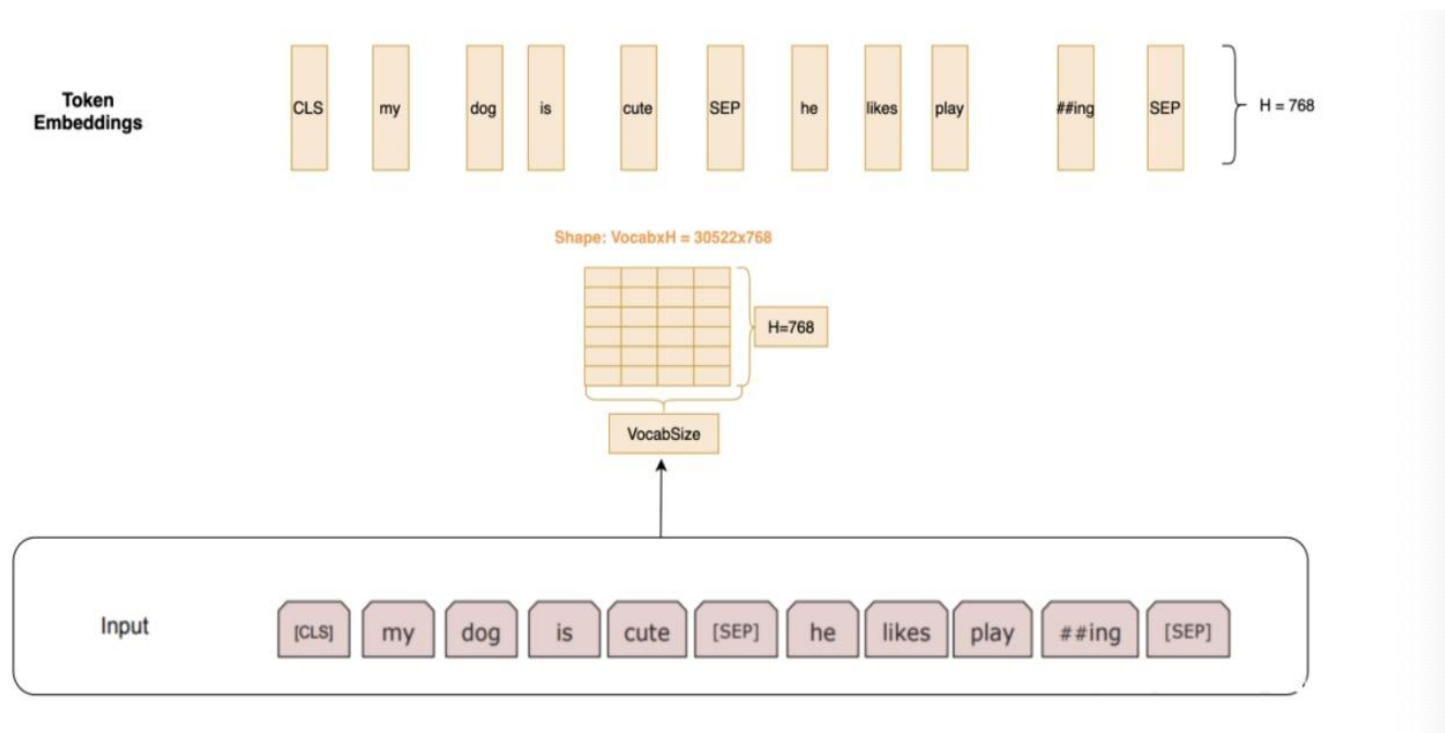
- BERT的输入是一个原始的文本序列，它可以是单个句子，也可以是两个句子（例如，问答任务中的问题和答案）。在输入到模型之前，这些文本需要经过特定的预处理步骤。

Tokenization 和 Embeddings

- **Tokenization:** 输入文本首先通过分词器 (Tokenizer) 被分割成Token。这一步通常包括将文本转换为小写、去除标点符号、分词等。BERT使用WordPiece分词方法，将单词进一步拆分成子词 (subwords)，以优化词汇表的大小和模型的泛化能力。
- **Token Embeddings:** 分词后的Token被映射到一个高维空间，形成Token Embeddings。这是通过查找一个预训练的嵌入矩阵来实现的，该矩阵为每个Token提供一个固定大小的向量表示。
- **Segment Embeddings:** 由于BERT能够处理两个句子作为输入（例如，在句子对分类任务中），因此需要一种方法来区分两个句子。Segment Embeddings用于此目的，为每个Token添加一个额外的嵌入，以指示它属于哪个句子（通常是“A”或“B”）。
- **Position Embeddings:** 由于Transformer模型本身不具有处理序列中Token位置信息的能力，因此需要位置嵌入来提供这一信息。每个位置都有一个独特的嵌入向量，这些向量在训练过程中学习得到。
- Token Embeddings、Segment Embeddings和Position Embeddings三者相加，得到每个Token的最终输入嵌入。

Token Embeddings

- 在BERT中，输入文本首先被分割成Token序列（单词、子词等），每个Token都会被映射到一个高维向量空间，形成Token Embeddings。这些嵌入捕获了Token的语义信息，是模型理解文本的基础。
- Token Embeddings是通过在大规模语料库上进行无监督预训练得到的，这使得BERT能够理解和处理各种复杂的语言现象和语义关系。



Position Embeddings:

- 由于Transformer结构本身不具备处理序列顺序的能力，BERT引入了Position Embeddings来捕获文本中Token的位置信息。
- 这些嵌入确保模型能够区分不同位置的相同Token，例如区分“hello, world”和“world, hello”中的“hello”和“world”。



BERT的网络结构

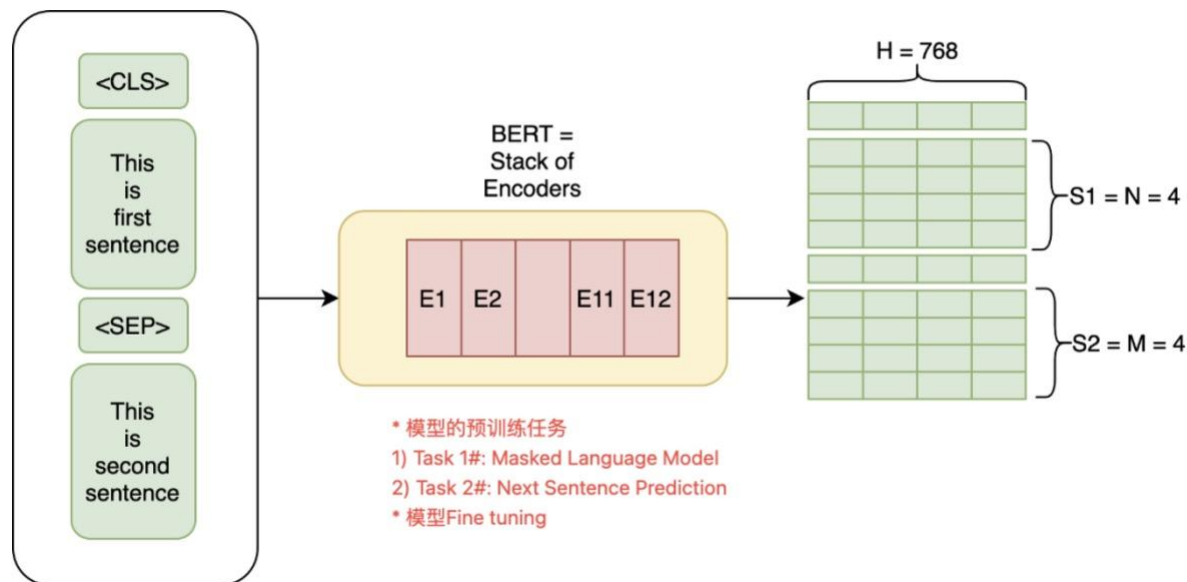
- BERT的核心是由多个Transformer编码器层堆叠而成的。每个编码器层都包含自注意力机制和前馈神经网络，允许模型捕捉输入序列中的复杂依赖关系。
- 自注意力机制: 允许模型在处理序列时关注不同位置的Token，并计算Token之间的注意力权重，从而捕捉输入序列中的依赖关系。
- 前馈神经网络: 对自注意力机制的输出进行进一步转换，以提取更高级别的特征。
- 残差连接和层归一化: 用于提高模型的训练稳定性和效果，有助于缓解梯度消失和梯度爆炸问题。

输出层

- BERT的输出取决于特定的任务。在预训练阶段，BERT采用了两种任务：Masked Language Model（MLM）和Next Sentence Prediction（NSP）。
- MLM: 在这种任务中，BERT预测输入序列中被随机遮盖的Token。模型的输出是每个被遮盖Token的概率分布，通过Softmax层得到。
- NSP: 这种任务要求BERT预测两个句子是否是连续的。模型的输出是一个二分类问题的概率分布。

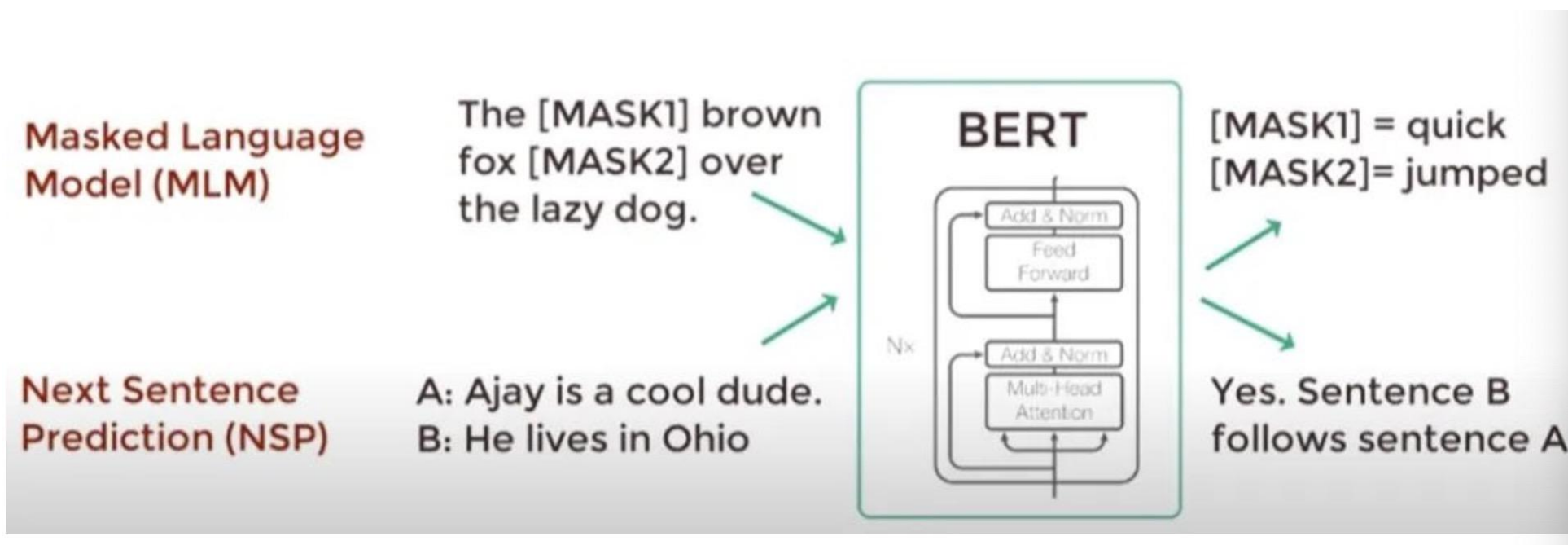
BERT 的原理

- BERT的工作原理是通过在大规模未标注数据上执行预训练任务（如Masked Language Model来捕获文本中词汇的双向上下文关系，以及Next Sentence Prediction来理解句子间的逻辑关系），再将预训练的模型针对特定任务进行Fine tuning，从而在各种自然语言处理任务中实现高性能。



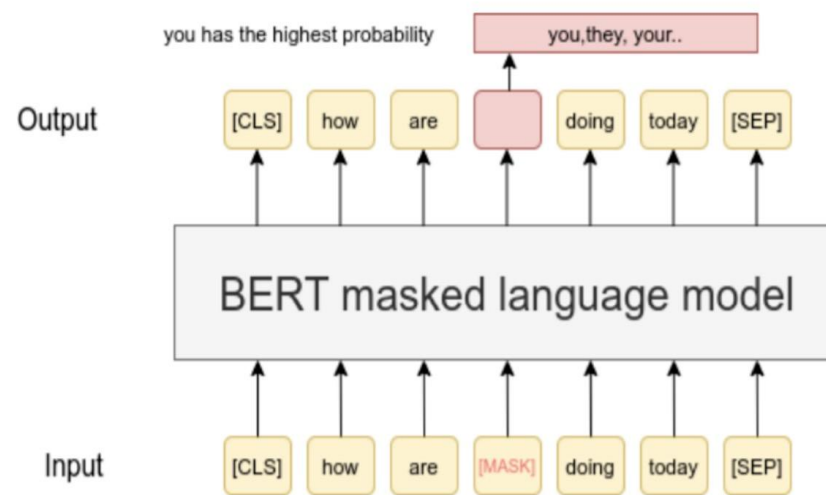
BERT工作原理

- 模型的预训练任务：在大量未标注数据上进行自我学习的过程，通过这些任务，模型能够学习到语言的内在规律和模式，从而为其在后续的具体任务（如文本分类、问答等）中提供有力的支持。以下是两个核心的预训练任务：



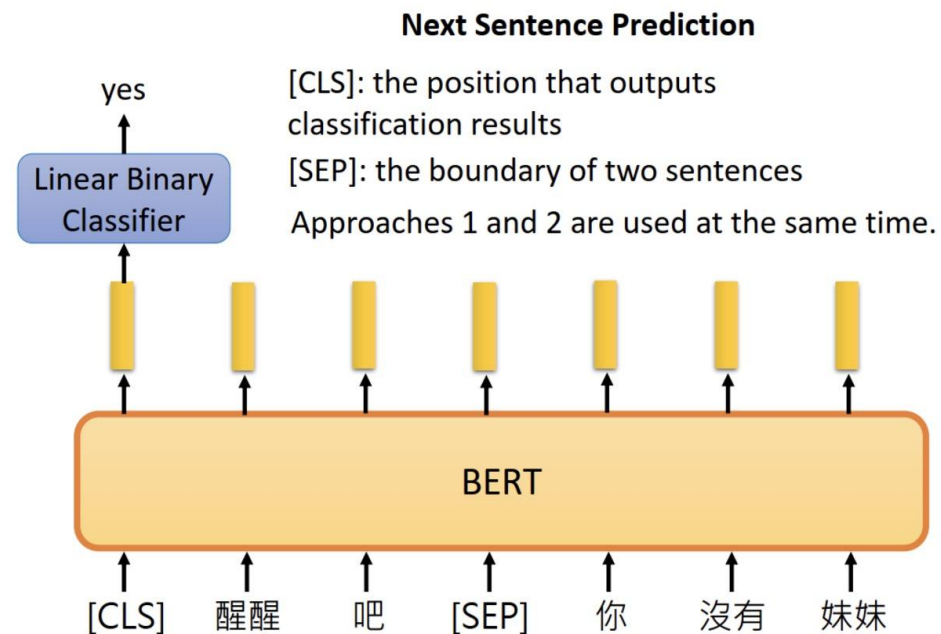
Masked Language Model (MLM)

- 任务描述：在输入的文本中，随机地遮盖或替换一部分词汇，并要求模型预测这些被遮盖或替换的词汇的原始内容。
- 目的：使模型能够利用双向的上下文信息来预测被遮盖的词汇，从而学习到更深层次的语义表示。
- 实现方式：在预训练阶段，BERT随机选择文本中15%的Token进行遮盖，其中80%的时间用[MASK]标记替换，10%的时间用随机词汇替换，剩下的10%保持不变。这种遮盖策略被称为动态遮盖，因为它在每次输入时都会随机改变遮盖的位置和词汇。



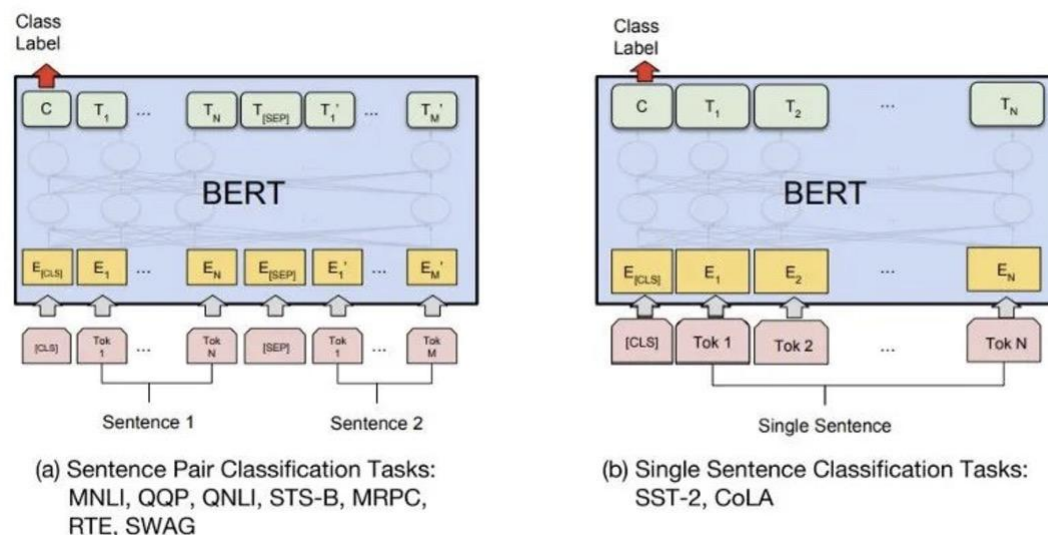
Next Sentence Prediction (NSP)

- 任务描述：给定一对句子，判断第二个句子是否是第一个句子的后续句子。
- 目的：使模型能够理解句子间的逻辑关系，如连贯性、因果关系等，从而提高其在处理长文档或复杂文本时的能力。
- 实现方式：在预训练阶段，BERT构造了一个二分类任务，其中50%的时间B是A的真正后续句子（标签为“IsNext”），另外50%的时间B是从语料库中随机选择的句子（标签为“NotNext”）。模型通过最后一层Transformer输出的[CLS]标记的嵌入来进行预测。



模型Fine tuning

- BERT的fine-tuning过程是针对特定任务对预训练模型进行调整的过程，使其能够更好地适应和解决具体任务。根据任务类型的不同，对BERT模型的修改也会有所不同，但通常这些修改都相对简单，往往只需要在模型的输出部分加上一层或多层神经网络。



模型Fine tuning

任务类型：根据具体任务的不同，BERT的fine-tuning可以分为以下几种类型：

- 句子对分类任务（Sentence Pair Classification）：这种任务需要判断两个句子之间的关系，如文本蕴含、问答匹配等。在fine-tuning时，将两个句子一起输入模型，并取第一个token（[CLS]）的输出表示作为整个句子对的表示，然后将其输入到一个额外的softmax层进行分类。
- 单句分类任务（Single Sentence Classification）：这种任务需要对单个句子进行分类，如情感分析、文本分类等。在fine-tuning时，将单个句子输入模型，并同样取第一个token（[CLS]）的输出表示进行分类。
- 问答任务（Question Answering）：这种任务需要模型从给定的文本中找出问题的答案。在fine-tuning时，将问题和答案一起输入模型，并取答案部分在模型输出中的起始和结束位置作为答案的预测。

模型Fine tuning

模型修改：在进行fine-tuning时，对BERT模型的修改通常包括以下几个方面：

- 输入处理：根据任务类型的不同，对输入数据进行相应的处理，如将句子对拼接在一起、添加特殊标记等。
- 输出层：在BERT模型的输出部分添加一层或多层神经网络，用于将模型的输出转换为任务所需的格式。对于分类任务，通常添加一个softmax层进行概率分布的计算；对于序列标注任务，则逐token进行分类。
- 损失函数：根据任务类型选择合适的损失函数，如交叉熵损失函数用于分类任务、平方差损失函数用于回归任务等。

